

第 27 章: Class Coding Basics (类编码基础)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: OOP 从抽象理论走向实际代码的第一章。本章用大量交互式示例讲解 class 语句的三大核心能力: 生成多个实例对象、命名空间继承、运算符重载, 并首次揭示 "类本质上就是带链接的命名空间字典" 这一底层真相。学完本章, 你就能读懂绝大多数 Python OOP 代码。

27.1 开篇引言

英文原文

Now that we've talked about OOP in the abstract, it's time to see how this translates to actual code. This chapter begins to fill in the syntax details behind the class model in Python. If you've never been exposed to OOP in the past, classes can seem somewhat complicated if taken in a single dose. To make class coding easier to absorb, we'll begin our detailed exploration of OOP by taking a first look at some basic classes in action in this chapter. We'll expand on the details introduced here in later chapters of this part of the book, but in their basic form, Python classes are easy to u

nderstand. In fact, classes have just three primary distinctions. At a base level, they are mostly just namespaces, much like the modules we studied in Part V. Unlike modules, though, classes also have support for generating multiple objects, for namespace inheritance, and for operator overloading. Let's begin our class statement tour by exploring each of these three distinctions in turn.

中文翻译

我们已经从抽象层面讨论了 OOP，现在是时候看看它如何转化为实际代码了。本章开始填补 Python 中 class（类）模型背后的语法细节。如果你以前从未接触过 OOP，一次性摄入太多内容，类可能会显得相当复杂。为了让类编码更容易消化吸收，本章我们将先初步观察一些基础类在实际运行中的表现，从而开始对 OOP 的详细探索。本书这一部分的后续章节会扩展这里引入的细节，但就基本形式而言，Python 类其实很容易理解。事实上，类只有三个主要的区别性特征。在最基本的层面上，它们大多只是命名空间（namespace），很像我们在第五部分学习的模块（module）。但与模块不同的是，类还支持生成多个对象、支持命名空间继承（inheritance）、支持运算符重载（operator overloading）。让我们逐一探索这三个区别，开始我们的 class 语句之旅。

深度理解

·核心概念：本章是 OOP 的“语法落地”章节。前面第 26 章讲的是 OOP 的抽象思想（为什么需要对象），本章回

答"在 Python 里到底怎么写"。作者给了全章一个总纲：类只有三个主要区别——多实例、继承、运算符重载；其余的一切（属性、方法、命名空间）你都已经在模块和函数里学过了。

·底层视角：注意作者反复用"命名空间"来定义类——这说明在 Python 的心智模型里，类不是一张"静态的图纸"，而是一个运行时存在、可以挂属性、可以被查找的对象。

·学习策略：作者特意用 "easy to understand" 给读者打气。事实上本章的代码量很少，核心就是三个例子（First Class、SecondClass、ThirdClass），建议你在自己的 REPL 里跟着敲一遍。

·生活例子：把"模块 vs 类"想成"一张全班共用的课程表 vs 每位学生的个人日程表"——模块只有一份，大家都看同一张；类可以按模板生成无数份各自不同的实例。

27.2 类生成多个实例对象 (Classes Generate Multiple Instance Objects)

英文原文

To understand how the multiple objects idea works, you have to first understand that there are two kinds of objects in Python's OOP model: class objects and instance objects. Class objects provide default behavior and are used to create instance objects. Instance objects are the tangible objects your programs process —each is a namespace in its own right but inherits (

i.e., has automatic access to) names in the class from which it was created. Class objects come from statements, and instances come from calls; each time you call a class, you get a new instance of that class.

This object-generation concept is very different from most of the other program constructs we've seen so far in this book. In effect, classes are essentially factories for generating multiple instances. By contrast, only one copy of each module is ever imported into a single program. In fact, this is why reload works as it does, updating a single instance and shared object in place. With classes, each instance can have its own independent data, supporting multiple versions of the object that the class models.

In this role, class instances are similar to the per-call state of the closure (a.k.a. factory) functions of Chapter 17, but this is a natural part of the class model, and state in classes is explicit attributes instead of implicit scope references. Moreover, this is just part of what classes do—they also support customization by inheritance, operator overloading, and multiple behaviors via methods. Hence, classes are a more complete programming tool, though OOP and functional programming are not mutually exclusive paradigms. We may combine them by using functional tools in methods, by coding methods that are themselves generators, by writing user-defined iterators, and so on.

The following is a quick summary of the bare essentials of Python OOP in terms of its two object types. As you'll see, Python classes are in some ways similar to both defs and modules, but they may be quite different from what you're used to in other languages.

中文翻译

要想理解"多对象"的想法如何运作，首先必须明白 Python 的 OOP 模型中有两类对象：类对象（class object）和实例对象（instance object）。类对象提供默认行为，并用于创建实例对象。实例对象是程序所处理的具体对象——每个实例本身就是一个命名空间，但它会继承（即自动获得访问权）创建它的类中的名字。类对象来自语句（statement），实例来自调用（call）；每次调用一个类，你都会得到该类的一个新实例。这种对象生成的概念，与我们迄今为止在本书中见到的大多数程序构造非常不同。实际上，类本质上就是生成多个实例的工厂（factory）。相比之下，每个模块在一个程序中只会被导入一份。事实上，这正是 reload（重新加载）之所以那样工作的原因——在原地更新单个实例和共享对象。而有了类，每个实例都可以拥有自己独立的数据，从而支持同一个类所建模对象的多个版本。在这种角色下，类实例类似于第 17 章中闭包（closure，又称工厂）函数的"每次调用各自的状态"，但这是类模型的自然组成部分；而且在类中，状态是显式的属性（attribute），而不是隐式的作用域引用。此外，这只是类能力的一部分——类还支持通过继承定制、运算符重载，以及通过方法（method）实现多种行为。因此，类是更完整的编程工具，尽管 OOP 和函数式编程（functional programming）

并不是互斥的范式。我们可以把它们结合起来：在方法中使用函数式工具、把方法本身写成生成器（generator）、编写用户自定义迭代器（iterator），等等。下面是以这两类对象为纲，对 Python OOP 最核心要点的快速总结。你会看到，Python 类在某些方面既像 def 又像模块，但它们可能与你熟悉的其他语言中的类相当不同。

深度理解

- 核心概念：OOP 模型里永远存在两类对象：类对象（由 class 语句产生）和实例对象（由"调用类"产生）。"类对象来自语句，实例来自调用"这句话是整个模型的总纲——它不是关于写法的，而是关于运行时机的。
- 底层实现：在 CPython 中，"调用类" `X = MyClass()` 实际上执行的是类对象的 call 协议（由元类 type 实现）：分配新对象、调用 new、必要时调用 init 初始化、把新实例的 class 指针指向类对象、最后返回实例。每次调用都走完整流程，所以每次都会得到全新的对象。
- 设计原因：模块天然单例（一次 import 只有一份），所以模块无法表达"同一实体的多个版本"。类把"共享部分"（类对象里的属性）和"独立部分"（实例各自的 dict）分开存放，这正是工厂模式（factory pattern）的本质。
- 对比第 17 章的闭包：闭包函数的"每次调用各自状态"靠的是隐式作用域引用（闭包单元），而类的"每实例状态"靠的是显式属性赋值。显式的好处是：属性可以被外部读取、修改、继承、序列化——闭包状态是"锁在函数肚子里"的。Lutz 特意点出这一点，是提醒你类不是函数式编

程的对立面，两者可以混用。

·初学者误区：很多从 C++/Java 过来的人以为"类是编译期的模板"。在 Python 中类是运行时对象，class 只是产生它的语句——没有执行到那条语句，类就不存在。同一个 REPL 里可以随时动态改类、给类挂新属性，这在静态语言里是不可想象的。

·生活例子：类对象就像奶茶店的配方本（一套做法，共享），实例就像按配方现做的每一杯奶茶（每杯的甜度、温度可以不同）。模块则像一本全员共用的值班表——全公司永远只有这一张。

27.3 类对象提供默认行为 (Class Objects Provide Default Behavior)

英文原文

When we run a class statement, we get a class object. Here's a rundown of the main properties of Python classes: - The class statement creates a class object and assigns it a name. Just like the function definition, the Python class is an executable statement. When reached and run, it generates a new class object and assigns it to the first name in the class header. Also, like defs, class statements typically run when the files they are coded in are first imported or run as a top-level script. - Assignments inside class statements make class attributes. Just like in module files, top-level

assignments within a class statement (not nested in a def) generate attributes in a class object. Technically, the

class statement defines a local scope that morphs into the attribute namespace of the class object, just like a module's global scope. After running a class statement, class attributes may be accessed by name qualification: `class.name`. - Class attributes provide object state and behavior. Attributes of a

class object record state information and behavior to be shared by all instances created from the class. Most notably and commonly, function

def statements nested inside a class generate methods, which process instances.

中文翻译

当我们运行一个 class 语句时，就得到了一个类对象。下面是对 Python 类主要性质的梳理：- class 语句创建类对象并给它赋予一个名字。就像函数 def 语句一样，Python 的 class 是一条可执行语句（executable statement）。当执行流到达并运行它时，它会生成一个新的类对象，并把它赋给 class 头行中的第一个名字。同样与 def 类似，class 语句通常会在其所在文件被首次导入、或以顶层脚本方式运行时执行。- class 语句内部的赋值会生成类属性（class attribute）。就像在模块文件中一样，class 语句内部顶层的赋值（不嵌套在 def 中）会在类对象中生成属性。严格来说，class 语句定义了一个局部作用域，这个作用域会蜕变（

morphs) 为类对象的属性命名空间，就像模块的全局作用域那样。运行完 class 语句后，可以通过名字限定 (name qualification) 访问类属性：class.name。- 类属性提供对象的状态和行为。类对象的属性记录了状态信息和行为，供从该类创建的所有实例共享。最显著也最常见的例子是：嵌套在 class 语句内的 def 函数语句生成方法 (method)，由它们来处理实例。

中文翻译 2 (补充说明)

以上三条是"类对象"的完整画像：① 类是执行产物不是声明；② 类体内的顶层赋值 = 类属性；③ 类属性是全体实例的"公共资源"。

深度理解

·核心概念：class 与 def 一样，是可执行语句 (executable statement) 而非声明。它平时躺在模块文件里，只有被 import 或作为顶层脚本执行时，才真正"跑"一遍并创建类对象。这是 Python "一切皆运行时"哲学在类上的体现。

·底层实现：CPython 执行 class Foo: 语句时，大致做三件事：(1) 创建一个新的命名空间字典 (类似模块字典)；(2) 以该字典为作用域，逐条执行类体内的语句 (这就是为什么 x = 1 这样的顶层赋值会把 x 放进类字典)；(3) 用 type (元类) 把字典包装成类对象，并把名字 Foo 绑定到它。所谓"局部作用域 morph 成属性命名空间"，说的正是第 (1)(2) 步——作用域是编译期的概念，属性命名空间是运行期的结果，二者是同一份字典的两副面孔。

·设计原因：把方法用普通 `def` 写在类体内，而不是像 Java 那样用专用语法声明，是为了保持语言最小化——方法就是函数，函数的一切能力（默认参数、返回值、`yield`、装饰器）自动适用于方法，无需新语法。

·实际问题：理解了“class 是可执行语句”，就能理解很多怪现象：为什么类可以定义在 `if` 分支里（条件成立才创建类）；为什么同一个名字的类可以在不同函数里定义出不同版本；为什么 class 也可以出现在函数内部（局部类）

。

·初学者误区：以为“类体里的变量是实例变量”——错！类体顶层赋值（`spam = 1`）创建的是类属性，所有实例共享同一份；要想每实例独立，必须通过方法里的 `self.spam = ...` 赋值。这是全书最常见的混淆点之一。

·生活例子：class 语句就像一条流水线的开机按钮——按下（执行）了才有机器和工人（类对象）；没按（没执行到）就什么都没有。类体里的顶层赋值则像公司墙上贴的“全体员工守则”，一份，人人可见。

27.4 实例对象是具体条目 (Instance Objects Are Concrete Items)

英文原文

When we call a class object, we get an instance object. Here's an overview of the key points behind class instances: - Calling a class object like a function make

s a new instance object. Each time a class is called, it creates and returns a new instance object. Instances represent material items in your program's domain. - Each instance object inherits class attributes and gets its own namespace. Instance objects created from classes are new namespaces; they start out empty but inherit attributes that live in the class objects

from which they were generated.- Assignments to attributes of self in methods make per-instance attributes. Inside a class's method functions, the first argument (called self by convention) references the instance object being processed; assignments to attributes of self create or change data in the instance, not the class.

The end result is that classes define common, shared data and behavior, and generate instances. Instances reflect palpable application entities and record perinstance data that may vary per object.

中文翻译

当我们像调用函数一样调用一个类对象时，就得到一个实例对象。下面是类实例的关键要点概览：- 像调用函数一样调用类对象，会生成一个新的实例对象。每次调用类，它都会创建并返回一个新的实例对象。实例代表了程序领域中的具体条目（material items）。- 每个实例对象继承类属性，并拥有自己的命名空间。由类创建的实例对象是新的命名空间；它们一开始是空的，但会继承生成它们的类对象中存放的属性。- 方法中对 self 属性的赋值，会生成每个实例各自

的属性。在类的方法函数内部，第一个参数（按惯例称为 `self`）引用正在被处理的实例对象；对 `self` 属性赋值会创建或改变实例中的数据，而不是类中的数据。最终结果是：类定义共同的、共享的数据与行为，并生成实例；实例反映具体的应用实体，并记录可能因对象而异的每实例数据。

深度理解

·核心概念：实例的三个要点可以浓缩成一句话——实例 = 一个空命名空间 + 一条指向类的链接 + `self` 赋值入口。实例没有自己的“说明书”，一切行为都从类那里继承；它只负责存放自己的数据。

·底层实现：CPython 中每个实例对象内部主要就是两个东西：`dict`（实例自己的属性字典，初始为空）和 `class`（指向类对象的指针）。属性访问 `x.attr` 的流程是：先查 `x.dict`，找不到就顺着 `x.class` 往上查。所谓“继承”就是这个自动搜索过程，不是复制粘贴。

·设计原因：为什么实例先空着？因为 Python 是动态语言——属性不预声明，第一次赋值才“诞生”。这让实例非常轻量、灵活，但也要求用 `__init__` 保证属性在构造时就被创建，否则后续方法调用可能因属性缺失而崩溃（本书后面第 28 章会展开）。

·实际问题：`self` 不是关键字，只是约定俗成的第一个参数名。Python 自动把“调用方法的实例”填进第一个参数位（`x.setdata('coding')` 实际执行 `FirstClass.setdata(x, 'coding')`）。这就是方法绑定（method binding）——函数本身还是那个函数，只是被包装后自动传入了第一个实

参。

·初学者误区：① 以为 self 是特殊语法——换成 this 或任意名字都行（但千万别换，约定即纪律）；② 以为"实例属性要提前声明"——不需要，直接赋值即可；③ 以为给 self 赋值等于给所有实例赋值——恰恰相反，它只作用于当前这个实例。

·生活例子：实例就像每个员工的工牌——工牌是空白的（空命名空间），佩戴者（self）是谁，上面就填谁的名字；而"工牌应该怎么挂、什么时候戴"这些规定（方法）印在公司的《员工手册》（类）里，大家共享一份。

27.5 第一个示例 (A First Example)

英文原文

Let's turn to a real example to show how these ideas work in practice. To begin, let's define a class named FirstClass by running a Python class statement interactively in any REPL:

```
python >> class FirstClass: # Define a class object
def setdata(self, value): # Define class's methods self.
    data = value # self is the instance
def display(self):
    print(self.data) # self.data: per instance
We're working interactively here, but typically, such a statement would be run when the module file it is coded in is imported. Like functions created with defs, this class won't even exist until Python reaches and runs this statement
```

nt.

Like all compound statements, the class starts with a header line that lists the

class name, followed by a body of one or more nested and (usually) indented statements. Here, the nested statements are defs; they define functions that implement the behavior the class means to export.

As we learned in Part IV, `def` is really an assignment. Here, it assigns function objects to the names `setdata` and `display` in the class statement's scope, and so generates attributes attached to the class—`FirstClass.setdata` and `FirstClass.display`. In fact, any name assigned at the top level of the class's nested block becomes an attribute of the class.

Functions inside a class are usually and traditionally called methods. They're coded with normal defs, and they support everything we've learned about functions already—they can have defaults, return values, yield items on request, and so on. But in a method function, the first argument automatically receives an implied instance object when called—the subject of the call. Let's create a couple of instances of our class to see how this works:

```
python >> x = FirstClass() # Make two instances >>
y = FirstClass() # Each is a new namespace
```

By calling the class this way (notice the parentheses),

we generate instance objects, which are just namespaces that have access to their classes' attributes. Properly speaking, at this point, we have three objects: two instances and a class. And really, we have three linked namespaces, as sketched in Figure 27-1. In OOP terms, we say that `x` "is a" `FirstClass`, as is `y`—they both inherit names attached to the class.

Figure 27-1. Classes and instances: namespaces in a class tree searched by inheritance The two instances start out empty but have links back to the class from which they were generated. If we qualify an instance with the name of an attribute that lives in the class object, Python fetches the name from the class by inheritance search (unless it also lives in the instance):

```
python >> x.setdata('coding') # Call methods: self is x
x >> y.setdata(3.14159) # Runs: FirstClass.setdata(y, 3.14159)
```

Neither `x` nor `y` has a `setdata` attribute of its own, so to find it, Python follows the link from instance to class. And that's about all there is to inheritance in Python: it happens at attribute qualification time, and it just involves looking up names in linked objects—here, by following the is-a links in Figure 27-1.

In the `setdata` function inside `FirstClass`, the value passed in is assigned to

`self.data`. Within a method, `self`—the name given to the leftmost argument by convention—automatically

refers to the instance being processed (x or y at this point), so the assignments store values in the instances' namespaces, not the class's. That's how the data names in Figure 27-1 are created.

Because classes can generate multiple instances, methods must go through the self argument to get to the instance to be processed. When we call the class's display method to print self.data, we see that it's different in each instance; on the other hand, the name display itself is the same in x and y, as it comes (is inherited) from the class:

```
python >> x.display() # Runs: FirstClass.display(x) coding >> y.display() # self.data differs in each instance
3.14159
```

Notice that we stored different object types in the data member in each instance—a string and a floating-point number. As with everything else in Python, instance attributes (sometimes called members) are not predeclared and have no type constraints; they spring into existence the first time they are assigned values, just like simple variables. In fact, if we were to call display on one of our instances before calling setdata, we would trigger an undefined name error—the attribute named data doesn't even exist in memory until it is assigned within the setdata method.

As another way to appreciate how dynamic this model is, consider that we can change instance attributes

either inside the class itself, by assigning to `self` in methods, or outside the class, by assigning to an explicit instance object:

```
python >> x.data = 'hacking' # Can get/set attributes
>> x.display() # Outside the class too hacking
```

Although less common, we could even generate an entirely new attribute in the instance's namespace by assigning to its name outside the class's method functions:

```
python >> x.anothername = 'apps' # Can set new attributes here too!
```

This would attach a new attribute called `anothername`, which may or may not be used by any of the class's methods, to the instance object `x`. Classes usually create all of the instance's attributes by assignment to the `self` argument, but they don't have to—programs can fetch, change, or create attributes on any objects to which they have references.

It usually doesn't make sense to add data that the class cannot use, and it's possible to prevent this with extra "privacy" code based on attribute-access operator overloading, as we'll discuss elsewhere in this book (in Chapters 30 and 39). Still, free attribute access translates to less syntax, and there are cases where it's even useful—for example, in coding data records of the sort we'll build later in this chapter.

中文翻译

让我们转向一个真实示例，看看这些想法在实践中如何运作。首先，我们在任意 REPL 中交互式地运行一条 Python `class` 语句，定义一个名为 `FirstClass` 的类：

```
python >>> class FirstClass: # 定义一个类对象
def setdata(self, value): # 定义类的方法
self.data = value # self 就是实例本身
def display(self): print(self.data) # self.data: 每个实例各有一份
我们这里在交互式环境下工作，但通常情况下，这样的语句会在其所在模块文件被导入时运行。与用 def 创建的函数一样，在 Python 执行到并运行这条语句之前，这个类根本不存在。与所有复合语句（compound statement）一样，class 以头行（header line）开头，头行列出类名，后面跟着一个或多个嵌套的、通常带缩进的语句体。这里嵌套的语句是 def；它们定义的函数实现了这个类想要对外输出的行为。正如我们在第四部分学到的，def 实际上就是一种赋值。在这里，它把函数对象赋给 class 语句作用域中的 setdata 和 display 这两个名字，从而生成了挂在类上的属性——FirstClass.setdata 和 FirstClass.display。事实上，凡是在类嵌套块的顶层赋值的任何名字，都会成为类的属性。类内部的函数通常（也是传统上）被称为方法（method）。它们用普通的 def 编写，支持我们已经学过的函数的一切特性——可以有默认参数、返回值、按需 yield 产出值，等等。但不同的是：在方法函数中，第一个参数在调用时会自动接收到一个隐含的实例对象——也就是这次调用的“主语”。让我们创建这个类的两个实例，看看它是如何工作的：
```

```
python >>> x = FirstClass() # 制作两个实例
>>> y = FirstClass() # 每个都是一个新命名空间
通过这样调用
```

类（注意括号），我们生成了实例对象——它们只是能够访问类属性的命名空间。严格来说，此时我们三个对象：两个实例和一个类。实际上，我们拥有三个相互链接的命名空间，如图 27-1 所示。用 OOP 的术语说，x 和 y "是一个" (is a) FirstClass——它们都继承了挂在类上的名字。图 27-1. 类与实例：继承搜索在类树中查找的命名空间两个实例开始时是空的，但它们都带有指回"生成自己的那个类"的链接。如果我们用实例去限定一个存在于类对象中的属性名，Python 会通过继承搜索 (inheritance search) 从类中取回该名字（除非实例自身也有这个名字）：

```
python >>> x.setdata('coding') # 调用方法：self 是 x >>> y.setdata(3.14159) # 实际执行：FirstClass.setdata(y, 3.14159)
```

x 和 y 都没有自己的 setdata 属性，所以要找到它，Python 就顺着"实例→类"的链接查找。Python 中的继承就这么多：它发生在属性限定时 (attribute qualification time)，无非是在链接起来的对象中查找名字——在这里就是沿着图 27-1 中的 is-a 链接。在 FirstClass 内部的 setdata 函数中，传入的值被赋给 self.data。在方法内部，self——按惯例给最左边参数起的名字——自动指向正在被处理的实例（此刻是 x 或 y），所以这些赋值把值存进了实例的命名空间，而不是类的。图 27-1 中的 data 名字就是这样创建的。因为类可以生成多个实例，方法必须通过 self 参数才能拿到"要处理的那个实例"。当我们调用类的 display 方法来打印 self.data 时，会看到它在每个实例中各不相同；另一方面，display 这个名字本身在 x 和 y 中是相同的——因为它来自（继承自）类：

```
python >>> x.display() # 实际执行：FirstClass.display(x) coding >>> y.display() # self.data 在每个实例中不同3.14159 注意我们在每个实
```

例的 data 成员里存放了不同类型的对象——一个字符串和一个浮点数。与 Python 中其他一切一样，实例属性（有时也叫成员，member）不需要预声明，也没有类型约束；它们在第一次被赋值时才“凭空出现”，就像普通变量一样。事实上，如果在调用 setdata 之前就对某个实例调用 display，会触发未定义名称错误——因为 data 这个名字在内存中根本不存在，直到它在 setdata 方法中被赋值。为了从另一个角度体会这个模型有多么动态，考虑：我们既可以在类内部通过给方法中的 self 赋值来修改实例属性，也可以在类外部通过给显式的实例对象赋值来修改：python >>> x.data = 'hacking' # 可以获取/设置属性>>> x.display() # 在类外部也可以hacking 虽然不太常见，我们甚至可以在类的方法函数之外，通过给实例命名空间中的名字赋值，生成一个全新的属性：python >>> x.anothername = 'apps' # 这里也可以设置新属性！这会给实例对象 x 挂上一个名为 anothername 的新属性，这个属性可能被类的某个方法用到，也可能永远不用。类通常通过给 self 参数赋值来创建实例的所有属性，但不是必须这样——只要程序持有某个对象的引用，就可以取、改、建这个对象上的任何属性。添加类无法使用的数据通常没有意义，而且借助基于属性访问的运算符重载的“私有性”代码，可以阻止这种情况——本书其他部分（第 30 章和第 39 章）会讨论。不过，自由的属性访问换来的是更少的语法开销，而且在某些场景下甚至很有用——比如本章后面要构建的那类数据记录（data record）。

深度理解

·核心概念：这个例子第一次完整展示了三件事——① cla

ss 语句创建类对象，类体内的 def 是给类挂属性；② 调用类得到实例，实例是"空命名空间 + 指向类的链接"；③ 方法通过 self 读写特定实例的数据。这三件事合起来就是 Python OOP 的"最小可用模型"。

·底层实现（方法绑定）：x.setdata('coding') 这一行，解释器实际做了两件事。第一步是属性查找：x 的 dict 里没有 setdata，于是沿 x.class 找到类对象里的函数对象；第二步是方法绑定：函数对象实现了描述符协议（descriptor protocol），get 会把函数包装成一个"绑定方法"（bound method）对象，悄悄把 x 记为该方法的第一个实参。所以最终调用等价于 FirstClass.setdata(x,'coding')。这就是"第一个参数自动收到实例"的底层机制。

·底层实现（class 语句）：class FirstClass: 执行时，CPython 创建一个空字典作为类体作用域，依次执行两个 def（每个 def 在字典里放入函数对象），最后调用 type(...) 创建类对象并绑定名字 FirstClass。类体的作用域"蜕变"成了类对象的 dict。

·设计原因：为什么实例属性不预声明、无类型约束？因为 Python 的属性就是字典里的键值对——字典的键是运行时才决定的。这一设计让对象像"可扩展的档案袋"，想装什么装什么，代价是读到一个从未赋过值的属性会直接报 AttributeError。

·实际问题：在类外改 x.data、甚至新增 x.anothername 都合法——这既是 Python 的灵活（少语法、适合数据记录），也是隐患（拼错属性名不会报错，只会静默创建一个新属性，产生诡异 bug）。Lutz 提前预告：第 30、39

章会讲如何用属性访问钩子做"私有性"约束。

·初学者误区：① 以为"每个实例会把类的方法复制一份"——没有，方法只有一份，实例通过链接共享；② 以为 `self.data = value` 是在改类——不是，它只改当前实例；③ 以为属性必须先声明——不需要，但必须先赋值再读取。

·生活例子：方法绑定就像共享单车——车（函数）只有一批，谁扫码（调用）就把"当前骑行者"（`self`）绑定到那趟行程上；扫一次绑一个人，互不干扰。

代码分析

```
>>> class FirstClass:           #
    def setdata(self, value):    #
        self.data = value       # self
    def display(self):           #
        print(self.data)        # data
```

·解释器如何处理：执行 `class` 头行后，解释器新建一个命名空间字典，然后逐行执行类体：两个 `def` 语句各创建了一个函数对象并存入该字典（`setdata`、`display` 两个键）。类体执行完毕，解释器用 `type()` 把字典包装成类对象，绑定到名字 `FirstClass`。此刻函数还没有被"包成方法"——它们只是类里的普通函数属性。

```
>>> x = FirstClass()           #
>>> y = FirstClass()           #
```

·解释器如何处理：调用 `FirstClass()` 触发类对象的调用协议，分配一个新实例对象（内部持有空的 `dict` 和指向 `FirstClass` 的 `class` 指针），返回后绑定给 `x`；再次调用又创

建另一个独立实例给 *y*。所以现在内存里是三个对象：*x*、*y*（两个空实例）+ *FirstClass*（一个有方法的类）。

```
>>> x.setdefault('coding')          # self  x
>>> y.setdefault(3.14159)           # FirstClass.setdefault(y, 3....
```

解释器如何处理：*x.setdefault('coding')* 分两步走：先在 *x* 的 *dict* 中找 *setdata*（没有），沿 *x.class* 找到类里的函数，触发绑定（把 *x* 记为第一个实参），然后以 ('coding') 为剩余实参调用它。进入函数后 *self.data = value* 是把键 *data* 写入 *x* 自己的 *dict*。同理，*y.setdefault(3.14159)* 把键 *data* 写入 *y* 的字典。两个实例各有各的 *data*，互不相干——这就是“每实例数据”的诞生过程。

```
>>> x.display()                      # FirstClass.display(x)
coding
>>> y.display()                      # self.data
3.14159
```

解释器如何处理：*display* 也是从类继承来的同一个函数，绑定到不同实例后读取各自的 *data* 并打印，所以输出不同。注意 *display* 这个名字本身只有一个副本，存在类里——实例里没有。

```
>>> x.data = 'hacking'               # /
>>> x.display()                      #
hacking
>>> x.anothername = 'apps'           #
```

解释器如何处理：赋值语句 *x.data = ...* 不做继承搜索——它只在 *x* 的字典里写键值（有则覆盖，无则新建）。所以类外改属性、类外加属性都是直接改字典，零限制。这也解释了 Python 数据模型里的铁律：读取属性才搜索（继承），写入属性只落在当前对象。这条规则在 27.11 节

会有更直观的演示。

27.6 类通过继承进行定制 (Classes Are Customized by Inheritance)

英文原文

Let's move on to the second major distinction of classes. Besides serving as factories for generating multiple instance objects, classes also allow us to make changes by introducing new components (called subclasses), instead of changing existing components in place.

As we've seen, instance objects generated from a class inherit the class's attributes. Python also allows classes to inherit from other classes, opening the door to coding hierarchies of classes that specialize behavior—by redefining attributes in subclasses that appear lower in the hierarchy, we override the more general definitions of those attributes higher in the tree. In effect, the further down the hierarchy we go, the more specific the software becomes. Here, too, there is no parallel with modules, whose attributes live in a single, flat namespace that is not as amenable to customization.

In Python, instances inherit from classes, and classes inherit from superclasses. Here are the key ideas behi

nd the machinery of attribute inheritance: - Superclasses are listed in parentheses in a class header. To make a

class inherit attributes from another class, just list the other class in parentheses in the new class statement's header line. The class that inherits is usually called a subclass, and the class that is inherited from is its superclass. - Classes inherit attributes from their superclasses. Just as instances inherit the attribute names defined in their classes, classes inherit all of the attribute names defined in their superclasses. Python finds these names automatically when they're accessed if they don't exist in the subclasses. - Instances inherit attributes from all accessible classes. Each instance gets names from the class it's generated from, as well as all of that class's superclasses. When looking for a name, Python checks the instance, then its class, then all superclasses above its class. - Each `object.attribute` reference invokes a new, independent search. Python performs an independent search of the class tree for each attribute fetch expression. This includes references to instances and classes made outside class statements (e.g., `X.attr`), as well as references to attributes of the `self` instance argument in a class's method functions. That is, each `self.attr` expression in a method invokes a new search for `attr` in `self` and above.

The net effect—and the main purpose of all this searching—is that classes support factoring and customiz

ation of code better than any other language tool we've seen so far. On the one hand, they allow us to minimize code redundancy (and so reduce maintenance costs) by factoring operations into a single, shared implementation; on the other, they allow us to program by customizing what already exists, rather than changing it in place or starting from scratch.

NOTE — Full inheritance disclosure: Strictly speaking, Python's inheritance is richer than described here, when we factor in "diamond" inheritance patterns and "metaclasses"—advanced topics we'll study later—but we can safely restrict our scope to instances and their classes, both at this point in the book and in most Python application code. We'll explore diamonds and the "MRO" inheritance search order that accommodates them in Chapter 31, but our definition of inheritance won't be fully complete until Chapter 40, because it regrettably requires metaclass info that's beyond almost all Python programmers' interest levels and pay grades (thankfully!).

中文翻译

接下来看类的第二个主要区别。除了作为生成多个实例对象的工厂之外，类还允许我们通过引入新的组件（称为子类，subclass）来做出修改，而不是原地改动已有组件。正如我们所见，由类生成的实例对象会继承类的属性。Python 还允许类继承其他类，从而打开了编写“专门化行为”的类层次（hierarchy）的大门——通过在层次中较低位置的子类里

重新定义属性，我们就能覆盖（override）树中较高位置的、更通用的定义。实际上，层次越往下，软件就越具体。在这方面，模块没有对应物：模块的属性活在单一、扁平的命名空间里，不容易定制。在 Python 中，实例继承类，类继承超类（superclass）。以下是属性继承机制的几个关键点：

- 超类列在 class 头行的括号里。要让一个类继承另一个类的属性，只需在新建的 class 语句头行括号中列出那个类。执行继承的类通常叫子类，被继承的类叫它的超类。
- 类继承超类的属性。正如实例继承其类中定义的属性名，类也继承其超类中定义的全部属性名。当这些名字在子类中不存在而被访问时，Python 会自动找到它们。
- 实例继承所有可到达类的属性。每个实例从生成它的类、以及该类的全部超类那里获得名字。查找一个名字时，Python 依次检查实例、它的类、再上溯到类之上的所有超类。
- 每次 object.attribute 引用都会发起一次全新的、独立的搜索。Python 对每一条属性获取表达式都独立执行一次类树搜索。这既包括在 class 语句之外对实例和类的引用（如 X.attr），也包括方法函数中对 self 实例参数的属性引用。也就是说，方法中的每一个 self.attr 表达式，都会为 attr 在 self 及其上层发起一次新搜索。净效果——也是所有这些搜索的主要目的——是：类比我们目前见过的任何语言工具都更支持代码的抽取（factoring）与定制。一方面，它允许我们把操作抽取进单个共享实现，从而最小化代码冗余（进而降低维护成本）；另一方面，它允许我们通过“定制已有的东西”来编程，而不是原地修改或从零开始。注意——继承的完整披露：严格地说，Python 的继承比这里描述的更丰富——如果把“菱形”（diamond）继承模式和“元类”（metaclass）这些高级主题考虑进来的话，我们到后面再学。但在本

书目前这个阶段，以及大多数 Python 应用代码中，把范围限制在“实例及其类”是安全的。我们将在第 31 章探讨菱形继承和与之配套的 MRO（方法解析顺序）继承搜索顺序，但继承的完整定义要到第 40 章才能讲完——因为很不幸，它需要元类的知识，而这超出了几乎所有 Python 程序员的兴趣范围和职位等级（谢天谢地！）。

深度理解

·核心概念：继承不是“复制”，而是“搜索时的优先级”。子类定义一个同名属性，并不是把超类的属性删掉，而是让自己在搜索路径上更靠前，于是“盖住”了超类的同名属性——这就是覆盖（override）。

·底层实现：继承搜索的物理实现就是“链表”：实例 `class` → 类 `bases`（一个元组，可含多个超类）→ 超类的 `bases`.....每条 `obj.attr` 表达式都从 `obj` 的 `dict` 开始，逐级向上查 `dict`，第一次命中即停。多个超类时按 MRO 顺序（菱形继承时是 C3 线性化）决定谁先谁后——本书第 31 章才展开。

·设计原因：为什么“每次引用都重新搜索”而不是缓存结果？因为属性可以被随时增删改（动态性），缓存会导致与运行时状态不一致。Python 宁可每次都做一次 $O(\text{深度})$ 的字典查找，也要保证“所见即所得”。类树通常很浅（深度 2-5 层），这个代价完全可以接受。

·实际问题：覆盖能力让你可以“在不触碰原代码的前提下扩展功能”——这是开放-封闭原则（open-closed principle）的语言级支持：对扩展开放（加子类），对修改封

闭（不动父类）。第三方库的类也照改不误，因为继承是外部的。

·初学者误区：① 以为子类会“继承到”父类实例的属性——不，实例属性永远属于实例，继承只发生在类属性层面；② 以为覆盖是“替换内存里的父类方法”——父类方法原封不动，只是查找时先撞见子类的版本；③ 以为继承必须声明——在 Python 中，不写括号的类隐式继承 object（27.12 节会看到 bases 里的它）。

·生活例子：继承就像快餐店的套餐进化——基础套餐（父类）保持不变，新推出的套餐（子类）“继承”原套餐的配菜，只把其中一两个单品换掉（覆盖）。老顾客点原套餐，口味完全不受影响；新套餐省去了重新设计全套配菜的功夫。

27.7 第二个示例 (A Second Example)

英文原文

To illustrate the role of inheritance, this next example builds on the previous one. First, we'll define a new class, `SecondClass`, that inherits all of `FirstClass`'s names and provides one of its own:

```
python >> class SecondClass(FirstClass): # Inherits setdata

def display(self): # Changes display print(f'Current value = "{self.data}")
SecondClass defines the display m
```

ethod to print with a different format. By defining an attribute with the same name as an attribute in FirstClass, SecondClass effectively replaces the display attribute in its superclass.

Recall that inheritance searches proceed upward from instances to subclasses to superclasses, stopping at the first appearance of the attribute name that it finds. In this case, since the display name in SecondClass will be found before the one in FirstClass, we say that SecondClass overrides FirstClass's display. Sometimes we call this act of replacing attributes by redefining them lower in the tree overloading.

The net effect here is that SecondClass specializes FirstClass by changing the behavior of the display method. On the other hand, SecondClass (and any instances created from it) still inherits the setdata method in FirstClass verbatim. Let's make a new instance to demonstrate:

```
python >> z = SecondClass() >> z.setdata('LP6e') # Finds setdata in FirstClass >> z.display() # Finds overridden method in SecondClass Current value = "LP6e"
```

As before, we make a SecondClass instance object by calling it. The setdata call still runs the version in FirstClass, but this time the display attribute comes from SecondClass and prints a custom message. Figure 27-2 sketches the namespaces involved.

Figure 27-2. Specialization: overriding inherited names by redefining them in subclasses Now, here's a crucial thing to notice about OOP: the specialization introduced in SecondClass is completely external to FirstClass. That is, it doesn't affect existing or future FirstClass objects, like the `x` from the prior example (assuming we're continuing the same REPL session):

```
python >> x.display() # x is still a FirstClass instance (old message) hacking
```

Rather than changing FirstClass, we customized it. Naturally, this is an artificial example, but as a rule, because inheritance allows us to make changes like this in external components (i.e., in subclasses), classes often support extension and reuse better than functions or modules can.

中文翻译

为了说明继承的作用，下一个示例建立在前一个示例之上。首先，我们定义一个新类 SecondClass，它继承 FirstClass 的所有名字，并提供一个自己的名字：python >>> class SecondClass(FirstClass): # 继承 setdata def display(self): # 改变 display print(f'Current value = "{self.data}") SecondClass 用不同的格式重定义了 display 方法。通过定义一个与 FirstClass 中同名的属性，SecondClass 实际上替换了其超类中的 display 属性。回想一下：继承搜索从实例向上到子类、再到超类，停在它找到属性名第一次出现的地方。在本例中，由于 SecondClass 中的 display 会比 FirstClass 中的先被找到，我们说 SecondClass 覆盖（

override) 了 FirstClass 的 display。有时我们把这“在树的更低位置重新定义以替换属性”的行为称为重载 (overloading)。这里的净效果是：SecondClass 通过改变 display 方法的行为，专门化 (specialize) 了 FirstClass。另一方面，SecondClass (以及从它创建的任何实例) 仍然原封不动地继承 FirstClass 中的 setdata 方法。让我们创建一个新实例来演示：python >>> z = SecondClass() >>> z.setdata('LP6e') # 在 FirstClass 中找到 setdata >>> z.display() # 在 SecondClass 中找到被覆盖的方法Current value = "LP6e" 和之前一样，我们通过调用 SecondClass 来创建实例对象。setdata 调用仍然运行 FirstClass 中的版本，但这次 display 属性来自 SecondClass，打印出自定义消息。图 27-2 勾勒了涉及的命名空间。图 27-2. 专门化：在子类中重新定义以覆盖继承的名字现在，关于 OOP 有一个至关重要的点要注意：SecondClass 引入的专门化对 FirstClass 而言是完全外部的。也就是说，它不会影响已存在或将来的 FirstClass 对象——比如前一个例子中的 x (假设我们还在同一个 REPL 会话中继续)：python >>> x.display() # x 仍然是 FirstClass 的实例 (旧消息) hacking 我们没有改变 FirstClass，而是定制了它。这当然是一个人为的示例，但作为一条规则：因为继承允许我们在外部组件 (即子类) 中做这类修改，类在扩展性和复用性上往往胜过函数或模块。

深度理解

·核心概念：本例演示了继承的两面性：复用 (setdata 原样继承) + 覆盖 (display 被替换)。class SecondClass (FirstClass): 括号里的 FirstClass 就是超类列表。

·底层实现：`z.display()` 的搜索路径是：`z.dict`（空）→ `SecondClass.dict`（找到 `display`，命中即停）。而 `z.setdata('LP6e')` 的路径是：`z.dict` → `SecondClass.dict`（没有 `setdata`）→ `FirstClass.dict`（命中）。两个方法一个取自已定义、一个取自父类，对调用者完全透明——调用者不知道（也不需要知道）方法定义在树的哪一层。

·设计原因：为什么覆盖要在“子类定义同名属性”而不是“修改父类”？因为修改父类会影响所有实例（包括已创建的 `x`），破坏性极大；而新增子类只影响新类及其实例。这就是“定制优于修改”的工程原则——最小影响面。

·实际问题：真实项目中，第三方库升级后你的子类照常工作，因为父类的旧行为被“盖住”而不是被改动；如果直接去改库代码，每次升级都要重打补丁。

·初学者误区：① 把 `override` 理解成“内存里替换方法”——没有替换，只是搜索次序；② 以为覆盖了 `display` 就必须重写 `setdata`——完全不用，没覆盖的照常继承；③ 忘记“同名的任何属性都会被覆盖”，不只是方法——数据属性同样适用。

·生活例子：父辈写的“通用请假流程”（父类），你在自己的部门（子类）里规定“请假必须附钉钉审批截图”（覆盖 `display`）——但“提前一天申请”（`setdata`）这条规矩照旧沿用。公司的原制度一个字没改，其他部门不受任何影响。

代码分析

```
>>> class SecondClass(FirstClass):      # setdata
    def display(self):                  # display
        print(f'Current value = "{self.data}")
```

·解释器如何处理：执行 class 语句时，解释器看到头行括号里有 FirstClass，于是把新类对象的 bases 设为 (FirstClass,)，并把 display 函数对象写入 SecondClass 自己的 dict。此刻类树是两层：FirstClass → SecondClass。

```
>>> z = SecondClass()
>>> z.setdata('LP6e')                # FirstClass setdata
>>> z.display()                      # SecondClass
Current value = "LP6e"
```

·解释器如何处理：z.setdata('LP6e')：查找 setdata 时，z 空、SecondClass 空、FirstClass 命中，于是执行 FirstClass.setdata 并把'LP6e'写入 z 的 dict。z.display()：查找 display 时在 SecondClass 自己的 dict 就命中了，根本不需要再往上走，所以执行的是新版本，打印 Current value = "LP6e"。f-string（格式化字符串）在函数执行时求值 self.data，取到 z 里的'LP6e'。

```
>>> x.display()                      # x FirstClass
hacking
```

·解释器如何处理：x 是 FirstClass 的实例，它的查找路径是 x.dict → FirstClass.dict，根本不经过 SecondClass，所以永远用旧版本 display，打印 hacking。这就是“专门化完全外部于 FirstClass”的机制原因——搜索路径决定了谁影响谁。

27.8 类是模块中的属性 (Classes Are Attributes in Modules)

英文原文

Before we move on, remember that there's nothing magic about a class name. It's just a variable assigned to an object when the class statement runs, and the object can be referenced with any normal expression. For instance, if our `FirstClass` were coded in a module file instead of being typed interactively, we could import it and use its name normally in a class header line:

```
python from modulename import FirstClass # Copy name into my scope
class SecondClass(FirstClass): # Use class name directly
def display(self): ... Or equivalently:
```

```
python import modulename # Access the whole module
class SecondClass(modulename.FirstClass): # Qualify to reference
def display(self): ... Like everything else, class names always live within a module, so they must follow all the rules we studied in Part V. For example, more than one class can be coded in a single module file—like other statements in a module, class statements are run during imports to define names, and these names become distinct module attributes. More generally, each module may arbitrarily mix any num
```

ber of variables, functions, and classes, and all names in a module behave the same way. The following hypothetical file demonstrates:

```
python # names.py var1 = 6 # names.var1 var2 = 3.12 # names.var2 def func1(): ... # names.func1 def func2(): ... # names.func2 class Cls1: ... # names.Cls1 class Cls2: ... # names.Cls2
```

This holds true even if the module and class happen to have the same name. For example, given the following imaginary file, person.py:

```
python class person: ...
```

we need to go through the module to fetch the class as usual:

```
python import person # Import module x = person.person() # Class within module
```

Although this path may look redundant, it's required: person.person refers to the person class inside the person module. Saying just person gets the module, not the class, unless the from statement is used:

```
python from person import person # Get class from module x = person() # Use class name
```

As with any other variable, we can never see a class in a file without first importing and somehow fetching it from its enclosing file. If this seems confusing, don't use the same name for a module and a class within it. In fact, common convention in Python recommends that class names should begin with an uppercase letter, and module names with a lowercase letter, to help make them more distinct (it's not required, but nearly common e

nough to be a rule):

```
python import person # Lowercase for modules x = p
erson.Person() # Uppercase for classes
```

Also, keep in mind that although classes and modules are both namespaces for attaching attributes, they correspond to very different source code structures: a module reflects an entire file, but a class is a statement within a file. We'll say more about such distinctions later in this part of the book.

中文翻译

在继续之前，请记住：类名没有任何魔法。它只是 `class` 语句运行时赋给一个对象的普通变量，这个对象可以用任何普通表达式来引用。比如，如果我们的 `FirstClass` 是写在模块文件里而不是在交互式环境中输入的，我们可以导入它，并在 `class` 头行中正常使用它的名字：`python from module name import FirstClass` # 把名字复制进我的作用域

```
class SecondClass(FirstClass): # 直接使用类名
def display(self): ...
```

或者等价地：`python import modulename` # 访问整个模块

```
class SecondClass(modulename.FirstClass): # 通过限定来引用
def display(self): ...
```

和其他一切一样，类名永远活在某个模块内部，因此必须遵守我们在第五部分学过的所有规则。例如，一个模块文件里可以编写多个类——和模块中的其他语句一样，`class` 语句在导入期间执行以定义名字，这些名字成为各自独立的模块属性。更一般地说，每个模块可以任意混用任意数量的变量、函数和类，模块中所有名字的行为方式完全相同。下面这个假设的文件可以说明：

```
python # names.py
var1 = 6 # names.var1
var2 =
```

3.12 # names.var2 def func1(): ... # names.func1 def func2(): ... # names.func2 class Cls1: ... # names.Cls1 class Cls2: ... # names.Cls2 即使模块和类恰好同名，这一规律依然成立。例如，给定下面这个虚构的文件 person.py：
python class person: ... 我们需要像往常一样穿过模块去取这个类：
python import person # 导入模块 x = person.person() # 模块内的类虽然这个路径看起来有些冗余，但它是必须的：
person.person 指的是 person 模块内部的 person 类。只说 person 拿到的是模块而不是类——除非使用 from 语句：
python from person import person # 从模块中取出类 x = person() # 使用类名与任何其他变量一样，不先导入、不设法从所在文件中取出，我们永远看不到一个文件中的类。如果这让你困惑，那就别让模块和它内部的类同名。事实上，Python 的常见约定是：类名以大写字母开头，模块名以小写字母开头，以便区分（并非强制，但已经近乎一条规则）：
python import person # 模块用小写 x = person.Person() # 类用大写另外请记住：虽然类和模块都是用来挂属性的命名空间，但它们对应截然不同的源代码结构——模块对应整个文件，而类只是文件里的一条语句。本书这一部分后面会更多地讨论这类区别。

深度理解

- 核心概念：类名 = 普通变量，类对象 = 普通对象。class 语句与 x = 5 在“给名字绑定对象”这件事上没有任何本质区别，只是绑定的对象是“类对象”而已。因此类可以出现在任何能放表达式的地方：导入、传参、放列表里都行。
- 底层实现：模块的 dict 与类的 dict 是同构的（都是字典）。import 语句执行模块文件，文件顶层的 class 语句依

次执行，类名进入模块字典。所以 `person.person` 就是"模块字典里取 `person` 键"。

·设计原因：Python 只有一种"组织单元"——模块（文件）。类不是独立的编译单元（不像 Java 的 `.class` 文件），它必须寄居在模块里。这让代码组织极度简单：一个文件里可以自由混合常量、函数、类，导入时按需取用。

·实际问题：`person.person()` 这种"模块名.类名"的冗余写法困扰过很多初学者，而约定（模块小写、类大写）正是为此而生。另外注意：`from module import Cls` 只是把名字复制进当前作用域，模块本身仍然只加载一份（`sys.modules` 缓存）。

·初学者误区：① 以为"import person 之后就可以直接用 `person()` 类"——不，要先 `person.person()` 或 `from` 导入；② 以为类定义必须独占文件——不需要；③ 以为 `import` 会"重新执行"模块里的类定义——同一模块只执行一次，之后都是缓存复用。

·生活例子：模块是小区，类是单元楼——地址总是"小区.楼号"（`person.Person`）。你从登记册（`from ... import ...`）把某栋楼的名字抄到自己的便签上，不代表小区里又建了一栋楼。

代码分析

```
from modulename import FirstClass      #
class SecondClass(FirstClass):          #
    def display(self): ...
```

```
import modulename                                #
class SecondClass(modulename.FirstClass):        #
    def display(self): ...
```

·解释器如何处理：两种方式最终都指向同一个类对象（模块只加载一份）。区别只在绑定方式：from 形式把名字直接绑定到当前作用域（相当于 FirstClass = modulename.FirstClass）；import 形式把模块对象绑定到 modulename 这个名字，用属性限定取用。class 头行括号里的表达式在类语句执行时求值一次，两种写法结果相同。

```
import person                                    #
x = person.person()                             #
```

```
from person import person                       #
x = person()                                    #
```

·解释器如何处理：import person 执行 person.py（其中的 class person: 创建类对象并写入模块字典），然后把模块对象绑定到名字 person。第一种写法中，person.person 两次属性查找取到类再调用；第二种写法把类直接绑定到当前作用域的名字 person 上，之后 person() 就是调用类。名字解析的规则完全一致——先导入，才能用。

27.9 类可以拦截 Python 运算符 (Classes Can Intercept Python Operators)

英文原文

Let's move on to the third and final major difference

between classes and modules: operator overloading. In simple terms, operator overloading lets objects coded with classes intercept and respond to operations that work on built-in types: addition, slicing, printing, qualification, and so on. It's mostly just an automatic dispatch mechanism—expressions and other built-in operations route control to implementations in classes. Here, too, there is nothing similar in modules: modules can implement function calls, but not the behavior of expressions (apart, that is, from the odd special case added in Python 3.7 for module `getattr` and `dir` functions covered in Chapter 25, and bemoaned there as a confusing conflation with classes—for reasons you're about to see for yourself).

Although we could implement all class behavior as normally named methods, operator overloading lets objects be more tightly integrated with Python's object model. Moreover, because operator overloading makes our own objects act like built-ins, it tends to foster object interfaces that are more consistent and easier to learn, and it allows class-based objects to be processed by code written to expect a built-in object's interface. Here is a quick rundown of the main ideas behind overloading operators:

- Methods named with double underscores (`__X__`) are special hooks. In Python classes, we implement operator overloading by providing specially named methods to intercept operations.

The Python language defines a fixed and unchangeable

ble mapping from each of these operations to a specially named method. - Such methods are called automatically when instances appear in built-in operations. For instance, if an instance object inherits an add method, that method is called whenever the object appears in a + expression. The method's return value becomes the result of the corresponding expression. - Classes may override most built-in type operations. There are dozens of special operator-overloading method names for intercepting and implementing nearly every operation available for built-in object types. This includes expressions, but also basic operations like printing and object creation. - Most operator-overloading methods have no default, and none are required. If a class does not define or inherit an operator-overloading method, it just means that the corresponding operation is not supported

for the class's instances. If there is no add, for example, + expressions raise exceptions. As you'll learn later, a root class named object that's an implicit superclass to every class does provide defaults

for some X methods, but not for many (e.g., object has a default for print strings, but not +). Importantly, operator overloading is an optional feature; it's used primarily by people developing tools for other Python programmers, not by application developers. And, candidly, you probably shouldn't use it just because it seems clever. Unless a class needs to mimic built-in

object interfaces, it should usually stick to nonoperator or method names whose calls are more explicit. Expressions like `and` and `+`, for example, may make sense for a numeric object like a matrix, but other code would generally serve its clients better with mnemonically named methods.

Because of this, we won't go into details on every operator-overloading method available in Python in this learner's book. Still, as previewed in the prior chapter, there is one operator-overloading method you are likely to see in almost every Python class: the `init` method, which is known as the constructor method and is used to initialize instance objects' state. Pay special attention to this method, because `init`, along with the `self` argument and inheritance search, turns out to be a core requirement for reading and understanding most OOP code in Python.

中文翻译

接下来看类与模块的第三个也是最后一个主要区别：运算符重载（operator overloading）。简单来说，运算符重载让用类编写的对象能够拦截并响应那些作用于内建类型（built-in type）的操作：加法、切片、打印、属性限定等等。它基本上就是一个自动分派（dispatch）机制——表达式和其他内建操作会把控制权路由到类中的实现。在这里，模块也没有对应物：模块可以实现函数调用，但无法实现表达式的行为（唯一例外是 Python 3.7 为模块 `getattr` 和 `dir` 函数添加的奇怪特例，在第 25 章讲过，并在那里被吐槽

为与类的令人困惑的混淆——原因你马上就会亲眼看到)。

虽然我们可以把所有类行为实现为普通命名的方法，但运算符重载让对象与 Python 的对象模型结合得更紧密。此外，因为运算符重载让自定义对象表现得像内建对象，它倾向于催生更一致、更容易学习的对象接口，也允许基于类的对象被那些"期望内建对象接口"的代码处理。下面是重载运算符的主要思想概览：

- 用双下划线命名的方法 (X) 是特殊的钩子 (hook)。在 Python 类中，我们通过提供特殊命名的方法来拦截操作，从而实现运算符重载。Python 语言定义了一个固定的、不可改变的映射，把每个操作对应到特定命名的方法。
- 当实例出现在内建操作中时，这些方法会被自动调用。例如，如果一个实例对象继承了 add 方法，那么每当该对象出现在 + 表达式中，这个方法就会被调用。方法的返回值成为对应表达式的结果。
- 类可以覆盖大多数内建类型操作。有几十个特殊的运算符重载方法名，用于拦截和实现内建对象类型可用的几乎所有操作。这包括表达式，也包括打印和对象创建等基础操作。
- 大多数运算符重载方法没有默认实现，也都不强制要求。如果某个类没有定义或继承某个运算符重载方法，那只是意味着该操作对这类实例不受支持。比如没有 add，+ 表达式就会抛出异常。

你以后会学到：名为 object 的根类是所有类的隐式超类，它为少数 X 方法提供了默认实现，但大多数都没有（例如 object 有打印字符串的默认实现，但没有 +）。重要的是：运算符重载是一个可选特性，主要被那些"为其他 Python 程序员开发工具"的人使用，而不是应用开发者。而且坦率地说，你不应该仅仅因为它看起来很酷就使用它。除非一个类需要模仿内建对象的接口，否则通常应该坚持使用调用更显式的、非运算符的方法名。比如和 + 这样的表达式对

矩阵这样的数值对象可能有意义，但其他代码通常用有助记名字的方法，对使用者更友好。正因如此，在这本入门书里，我们不会逐一讲解 Python 中所有可用的运算符重载方法。不过，正如上一章预告的那样，有一个运算符重载方法几乎会在每个 Python 类里出现：init 方法——它被称为构造函数（constructor）方法，用来初始化实例对象的状态。请特别关注这个方法，因为 init 连同 self 参数和继承搜索，正是阅读和理解大多数 Python OOP 代码的核心要件。

深度理解

·核心概念：运算符重载 = 表达式分派。a + b 在 Python 中不是“符号上的魔法”，而是被翻译成 type(a).add(a, b) 之类的方法调用。类只要定义了对应名字的钩子方法，就能接管该操作。

·底层实现：CPython 的求值循环（评估栈）遇到 + 操作码 BINARYOP 时，会查找左操作数的类型（及其基类链）里的 add 特殊方法；如果左操作数不支持，还会尝试右操作数的 radd（反向运算）。这个查找用的是类型而非实例本身（特殊方法查找在类型层级上进行，不经过实例字典——这是与普通属性查找的重要差异）。

·为什么这样设计：让 +、print()、len()、for 循环这些“语言级操作”统一走方法分派，意味着自定义对象可以无缝融入语言生态——任何为内建对象写的代码（排序、拼接、格式化）都能直接处理你的对象。这就是“协议”（protocol）思想：不需要继承某个具体基类，只要实现接口（duck typing 的运算符版）。

·实际问题：init 是唯一几乎必须用的重载方法——它不是为"炫技"，而是为"构造时初始化状态"，属于工程必需品。判断一个类是否该重载运算符的标准是：这个操作符对你的对象是否有直觉意义（矩阵用 + 合理，用户对象用 + 就莫名其妙）。

·初学者误区：① 以为 X 是保留字——不是，它们是普通名字，只是约定；② 以为不定义 add 时 + 会"自动做点什么"——不会，直接 TypeError；③ 以为 init 是"分配内存"的构造器——不是！分配内存由 new 负责，init 只做初始化，此时对象已经存在。本书第 30 章会细讲。

·生活例子：运算符重载像电话分机——语言只认"总机号码"（+ 这个符号），总机把电话自动转给分机（add 方法）。没有分机的部门（没定义方法的类），来电（表达式）就无人接听（抛异常）。

27.10 第三个示例 (A Third Example)

英文原文

On to another example. This time, we'll define a subclass of the prior section's SecondClass that implements three specially named attributes that Python will call automatically: - init is run when a new instance object is created: self is the new ThirdClass object.¹ - add is run when a ThirdClass instance appears in a + expression. - str is run when an object is printed (technically, when it's converted to its print string by the str b

uilt-in function or its Pythoninternals equivalent).

Our new subclass also defines a normally named method called `mul`, which changes the instance object in place. Here's the new subclass (copy-and-pasters: in REPLs, you may need to omit blank lines added here for clarity):

```
python >> class ThirdClass(SecondClass): # Inherit from SecondClass

def init(self, value): # On "ThirdClass(value)" self.data = value
def add(self, other): # On "self + other" return ThirdClass(self.data + other)
def str(self): # On "print(self)", "str()" return f'[ThirdClass: {self.data}]'
def mul(self, other): # In-place change: named self.data = other

ThirdClass "is a" SecondClass, so its instances inherit the customized display method from SecondClass of the preceding section. This time, though, ThirdClass creation calls pass an object to the value argument in the init constructor, where it is assigned to self.data. The net effect is that ThirdClass arranges to set the data attribute automatically at construction time, instead of requiring later setdata calls:
```

```
python >> a = ThirdClass(3) # init called
>> a.display() # Inherited method called
Current value = "3"
```

ThirdClass objects can also now show up in `+` expressions and print calls. For `+`, Python passes the instance object on the left to the `self` argument in `add` and the value on the right to `other`, as illustrated in Figure

27-3; whatever add returns becomes the result of the + expression (more on its result in a moment):

```
python >> b = a + 3 # add: makes a new instance >
> b.display() # b has all ThirdClass methods Current v
alue = "6"
```

For print, Python passes the object being printed to self in str; whatever string this method returns is taken to be the print string for the object. With str (or its broader twin repr, which we'll leverage in the next chapter), we can use a normal print to display objects of this class, instead of calling the display method. As a contrast, the added mul method also changes the instance in place for a named call:

```
python >> print(b) # str: returns display string [Third
Class: 6] >> a.mul(3) # mul: changes instance in place
>> print(a) # Print this instance's data [ThirdClass: 9]
```

Figure 27-3. Operator overloading: class methods are run for operators and operations Specially named methods such as init, add, and str are inherited by subclasses and instances, just like any other names assigned in a class. If they're not coded in a class, Python looks for such names in all its superclasses, as usual. As for all attributes, the lowest (most specific) version is used.

Operator-overloading method names are also not built-in or reserved words; they are just attributes that Python looks for when objects appear in various contexts.

exts. Python usually calls them automatically, but they may occasionally be called by your code as well. For example, the `init` method is often called manually to trigger required initialization steps in a superclass constructor, as you'll see in the next chapter.

返回结果——或者不 (Returning results—or not)

Some operator-overloading methods like `str` require results, but others are more flexible. For example, notice how the `add` method makes and returns a new instance object of its class, by calling `ThirdClass` with the result value—which in turn triggers `init` to initialize the result. This is a common convention, and explains why `b` in the listing has a `display` method; it's a `ThirdClass` object too, because that's what `+` returns for this class's objects. This essentially propagates the object type.

By contrast, `mul` changes the current instance object in place, by reassigning the `self` attribute. We could overload the expression to do the same with `mul`, but this would be too different from the behavior of for built-in types such as numbers and strings, for which it always makes new objects. Per common practice, overloaded operators should work the same way that built-in operations do. Because operator overloading is really just an expression-to-method dispatch mechanism, though, you can interpret operators any way you like in your own classes. Also, stay tuned for Chapter

30's related coverage of in-place operator methods (preview: imul handles =).

其他运算符重载方法 (Other operator-overloading methods)

Although we won't cover every operator-overloading method in this book, we'll survey additional common operator-overloading techniques in Chapter 30.

Again, while this is an optional tool that doesn't apply to most application programs, str is not uncommon, and the init constructor method is a norm that will be present in most Python classes you'll come across. In fact, even though instance attributes need not be predeclared in Python, you can usually find out which attributes an instance will have by inspecting its class's init method.

中文翻译

继续下一个示例。这次，我们定义上一节 SecondClass 的一个子类，实现三个 Python 会自动调用的特殊命名属性：

- 创建新实例对象时运行 init：此时 self 就是新的 ThirdClass 对象。¹- 当 ThirdClass 实例出现在 + 表达式中时运行 add。
- 当对象被打印时运行 str（严格说是当它被 str 内建函数或其 Python 内部等价物转换为打印字符串时）。我们的新子类还定义了一个普通命名的方法 mul，它原地修改实例对象。下面是新子类（复制粘贴的读者请注意：在 REPL 里可能需要删掉这里为清晰起见添加的空行）：

```
python >
>> class ThirdClass(SecondClass): # 继承 SecondClass
```

```

def init(self, value): # 在 "ThirdClass(value)" 时运行self
.data = value def add(self, other): # 在 "self + other"
时运行return ThirdClass(self.data + other) def str(self):
# 在 "print(self)"、"str()" 时运行return f'[ThirdClass: {s
elf.data}]'def mul(self, other): # 原地修改：命名方法self
.data = other ThirdClass "是一个" SecondClass，所以
它的实例继承上一节中 SecondClass 定制的 display 方法
。不过这次，创建 ThirdClass 的调用会把一个对象传给 ini
t 构造函数的 value 参数，在那里它被赋给 self.data。净
效果是：ThirdClass 在构造时自动设置好 data 属性，而不
再需要之后调用 setdata: python >>> a = ThirdClass(3
) # 调用 init >>> a.display() # 调用继承来的方法Curren
t value = "3" ThirdClass 对象现在也可以出现在 + 表达式
和 print 调用中。对于 +，Python 把左边的实例对象传给
add 的 self 参数，把右边的值传给 other，如图 27-3 所
示；add 返回的任何东西都成为 + 表达式的结果（其结果
稍后再细说）：python >>> b = a + 3 # add: 生成一个
新实例>>> b.display() # b 拥有 ThirdClass 的全部方法C
urrent value = "6" 对于 print，Python 把被打印的对象
传给 str 的 self；这个方法返回的任何字符串都被当作该对
象的打印字符串。有了 str（或其更宽泛的孪生兄弟 repr，
我们下一章会用到），我们可以用普通的 print 来显示这个
类的对象，而不必调用 display 方法。作为对比，新增的
mul 方法则是通过命名调用原地修改实例：python >>> p
rint(b) # str: 返回显示字符串[ThirdClass: 6] >>> a.mul
(3) # mul: 原地修改实例>>> print(a) # 打印该实例的数
据[ThirdClass: 9] 图 27-3. 运算符重载：运算符和操作会
运行类的方法 init、add、str 这类特殊命名的方法，与类

```

中赋值的其他任何名字一样，会被子类 and 实例继承。如果某个类没有编写这些名字，Python 会像往常一样在它的所有超类中查找。和所有属性一样，取最低（最具体）的版本。运算符重载的方法名也不是内建关键字或保留字；它们只是 Python 在对象出现在各种上下文中时会去查找的属性。Python 通常自动调用它们，但你的代码偶尔也可以手动调用。例如，init 方法经常被手动调用，以触发超类构造函数中必需的初始化步骤——你会在下一章看到。

返回结果——或者不 (Returning results—or not)

有些运算符重载方法（如 str）要求返回结果，而另一些更灵活。例如，注意 add 方法是怎样通过用结果值调用 ThirdClass，来制作并返回该类的一个新实例对象——这反过来又触发 init 来初始化结果。这是一个常见惯例，也解释了为什么清单中的 b 有 display 方法：它也是一个 ThirdClass 对象，因为 + 对这个类的对象返回的就是它。这实际上传播（propagate）了对象类型。相比之下，mul 通过重新赋值 self 的属性来原地修改当前实例。我们可以用 mul 重载表达式来做同样的事，但这与内建类型（如数字和字符串）的行为差异太大——内建类型总是制造新对象。按惯例，重载的运算符应当与内建操作的语义一致。不过，因为运算符重载本质上只是“表达式到方法”的分派机制，在自己的类里，你完全可以按自己喜欢的方式解释运算符。另外，敬请期待第 30 章关于就地运算符（in-place operator）方法的相关内容（预告：imul 处理 =）。

其他运算符重载方法 (Other operator-overloading methods)

虽然本书不会覆盖每一个运算符重载方法，但我们会在第 30 章调查更多常见的运算符重载技巧。再说一遍：虽然这是一个对大多数应用程序都不适用的可选工具，但 `str` 并不罕见，而 `init` 构造函数方法则是一种规范——你遇到的大多数 Python 类里都会有它。事实上，即使 Python 中的实例属性不需要预声明，你通常也可以通过查看类的 `init` 方法，得知一个实例将会具有哪些属性。

深度理解

·核心概念：第三个示例把“运算符重载”三件套凑齐了：`init`（构造时初始化）、`add`（+ 表达式）、`str`（打印）。注意它们的共同点——都是普通属性，与其他方法无异，只是名字碰巧被语言“约定”了。

·底层实现（`init` 的调用链）：`ThirdClass(3)` 的完整流程是：① 调用类对象 → 元类 `type.call`；② 调用 `ThirdClass.new`（默认从 `object` 继承）分配并返回一个空的实例；③ 若返回的是 `ThirdClass` 的实例，自动调用 `init(实例, 3)`；④ 返回实例。所以 `init` 里 `self.data = value` 是在“对象已存在、只是状态未初始化”时写入初始状态。

·底层实现（+ 的分派）：`a + 3` 在求值栈上执行 `BINARY OP` 指令，按类型层级查找 `add`；找到后调用 `ThirdClass.add(a, 3)`，返回值 `ThirdClass(6)` 成为表达式结果。注意 `add` 内部又调用了 `ThirdClass(...)`，于是递归触发了一次完整的构造流程——`b` 是一个全新的、`data=6` 的 `ThirdClass` 实例。

·设计原因：为什么 `add` 返回新对象而 `mul` 原地修改？因

为语言内建语义如此： $3 + 5$ 产生新整数， $a + b$ （字符串）也产生新字符串——“加法不破坏操作数”是用户心智预期；而 `mul` 是普通方法，行为由你定义。遵循内建语义，你的类才不会吓到使用者。

·实际问题：`print(b)` 能打出 `[ThirdClass: 6]`，靠的是 `str` 返回字符串。没有 `str` 时 `print` 会退而使用 `object` 的默认实现，输出 `<main.ThirdClass object at 0x...>`——这就是为什么日志和调试输出里“类都该有 `repr/str`”是工程常识。

·初学者误区：① 以为 `init` 是“分配内存”——不是，那是 `new` 的活，初学者完全不用碰 `new`；② 以为特殊方法“不能手动调用”——可以，只是不常见；③ 以为 $a + 3$ 会修改 `a`——不会，`add` 返回新对象，`a` 保持原样；④ 把 `str`（给人看的打印）和 `repr`（给调试者的、无歧义表示）混为一谈——本书下一章会区分。

·生活例子：`init` 像新员工入职登记——人已经来了（对象已分配），填表建档（初始化属性）；`add` 像称重台——把两筐苹果放上去，得到一个新的总重数字，两筐原苹果分毫未动；`str` 像商品标签——顾客（`print`）看到的就是你贴上去的那行字。

代码分析

```
>>> class ThirdClass(SecondClass):      # SecondClass
    def __init__(self, value):          # "ThirdClass(value)"
        self.data = value              #
    def __add__(self, other):           # "self + other"
        return ThirdClass(self.data + other) #
    def __str__(self):                  # "print(self)" "str()" ...
        return f'[ThirdClass: {self.data}]' #
    def mul(self, other):               #
        self.data *= other             #
```

·解释器如何处理：ThirdClass(3) 执行时：元类协议分配空实例 → 自动调用 init(新实例, 3) → self.data = 3 写入实例字典。a.display() 的搜索路径：a 空 → ThirdClass 无 display → SecondClass 命中（继承生效），打印 Current value = "3"。a + 3：求值栈对 + 执行特殊方法查找，命中 ThirdClass 的 add，调用 add(a, 3)，内部 self.data + other 得到整数 6，再 ThirdClass(6) 递归构造新实例（触发又一次 init），返回它作为表达式结果，绑定给 b。print(b)：print 会调用 str(b)，解释器在类型层级找 str，用其返回值作为打印内容。a.mul(3)：普通方法调用，self.data = other 等价于 self.data = self.data * other，直接改写 a 实例字典中的 data (3→9)。

```
>>> b = a + 3                                # __add__
>>> b.display()                             # b ThirdClass
Current value = "6"
```

·解释器如何处理：注意 b 与 a 是两个不同的对象——+ 没有改动 a (a.data 仍是 3)，而是把新实例交给 b。这是“加法不破坏操作数”语义的直接体现。b.display() 同样沿继承链找到 SecondClass 的 display。

```
>>> a.mul(3)                                # mul
>>> print(a)                                #
[ThirdClass: 9]
```

·解释器如何处理：mul 与方法调用毫无特殊之处：绑定 a 到 self，self.data = 3 把 a 自己的 data 从 3 改成 9。随后 print(a) 经 str 输出 [ThirdClass: 9]。对比 + 与 mul：一个"新生"，一个"原地"——这正是重载运算符时应遵循内建语义的直观教材。

27.11 世界上最简单的 Python 类 (The World's Simplest Python Class)

英文原文

Despite the details of the class statement that we've begun to uncover in this chapter, you should keep in mind that the basic inheritance model behind classes is very simple—all it really involves is searching for attributes in trees of linked objects. In fact, we can create a class with nothing in it at all. The following statement makes a class with no attributes attached, an empty namespace object:

```
python >> class rec: pass # Empty namespace object
```

We need the no-operation pass placeholder statement (discussed in Chapter 13) here because we don't have any methods to code. After we make the class by running this statement interactively, we can start attaching

ching attributes to the class by assigning names to it completely outside of the original class statement:

```
python >> rec.name = 'Pat' # Just objects with attributes  
>> rec.age = 40
```

And, after we've created these attributes by assignment, we can fetch them with the usual syntax. When used this way, a class is roughly similar to a "struct" in C, or a "record" in Pascal. It's basically an object with field names attached to it, and may be easier to code than alternatives like dictionaries:

```
python >> print(rec.name) # Like a C struct or a record  
Pat
```

Notice that this works even though there are no instances of the class yet; classes are objects in their own right, even without instances. In fact, they are just self-contained namespaces; as long as we have a reference to a class, we can set or change its attributes anytime we wish. Watch what happens when we do create two instances, though:

```
python >> x = rec() # Instances inherit class names  
> y = rec()
```

These instances begin their lives as completely empty namespace objects. Because they remember the class from which they were made, though, they will obtain the attributes we attached to the class by inheritance:

```
python >> x.name, y.name # name is stored on the class only ('Pat','Pat')
```

Really, these instances have no attributes of their own; they simply fetch the name attribute from the class object where it is stored. If we do assign an attribute to an instance, though, it creates (or changes) the attribute in that object, and no other—crucially, attribute references kick off inheritance searches, but attribute assignments affect only the objects in which the assignments are made.

Here, this means that `x` gets its own name, but `y` still inherits the name attached to the class above it:

```
python >> x.name = 'Sue' # But assignment changes x only >> rec.name, x.name, y.name ('Pat','Sue','Pat')
```

中文翻译

尽管本章已经揭示了不少 `class` 语句的细节，但请记住：类背后的基本继承模型非常简单——它实际上只涉及在链接对象的树中搜索属性。事实上，我们可以创建一个什么都没有的类。下面这条语句会创建一个不带任何属性的类，即一个空的命名空间对象：`python >>> class rec: pass` # 空的命名空间对象这里需要用到空操作占位语句 `pass`（第 13 章讨论过），因为我们没有任何方法要写。交互式运行这条语句创建类之后，我们可以完全在原始的 `class` 语句之外，通过给类赋值来开始挂接属性：`python >>> rec.name = 'Pat'` # 只是带属性的对象 `>>> rec.age = 40` 用赋值创建好

这些属性后，我们就可以用通常的语法获取它们。这样使用时，类大致类似于 C 语言的 "struct"（结构体）或 Pascal 语言的 "record"（记录）——本质上就是一个挂着字段名（field name）的对象，而且可能比字典之类的替代品更容易编写：

```
python >>> print(rec.name) # 像 C 的 struct 或记录 Pat
```

注意：即使这个类还没有任何实例，这一切也能工作——即使没有实例，类本身也是合法的对象。事实上，它们就是自包含的命名空间；只要持有某个类的引用，我们就可以随时设置或修改它的属性。不过，看看当我们真的创建两个实例时会发生什么：

```
python >>> x = rec() # 实例继承类的名字
>>> y = rec() # 这两个实例以完全空的命名空间对象开始它们的生命。
```

但正因为它们记得"自己由哪个类生成"，它们会通过继承获得我们挂在类上的属性：

```
python >>> x.name, y.name # name 只存储在类上 ('Pat', 'Pat')
```

实际上，这些实例自己没有属性；它们只是从存储 name 的类对象那里取回它。不过，如果我们真的给某个实例赋值一个属性，它只会在那个对象中创建（或改变）属性，而不会影响其他对象——关键在于：属性引用会触发继承搜索，但属性赋值只影响被赋值的对象本身。在这里，这意味着 x 得到了自己的 name，而 y 仍然继承挂在类上的 name：

```
python >>> x.name = 'Sue' # 但赋值只改变 x
>>> rec.name, x.name, y.name ('Pat', 'Sue', 'Pat')
```

深度理解

·核心概念：本章最重要的规则浓缩在这句话里——读取走继承，写入不传播（references search, assignments don't）。`x.name = 'Sue'` 不会修改类上的 name，而是把 'Sue' 写进 x 自己的字典，从此 x 的读取路径在实例这一

层就命中了，不再向上。y 与 rec 的 name 原封不动。

·底层实现：class rec: pass 执行后，rec.dict 是 {}（只有 dunder 内建键）；rec.name = 'Pat' 直接往类字典里加键。x = rec() 创建实例，x.dict 为空，class 指向 rec。x.name 的查找：x 字典没有 → 沿链接到 rec 字典 → 命中 'Pat'。x.name = 'Sue' 则只写 x 字典（新增键）。一次赋值、两个查找路径，机制就这么简单。

·设计原因：为什么赋值不继承？如果赋值也“继承”（写进类），那么改一个实例的数据会污染所有实例，多实例的独立性就荡然无存——那与共享全局变量无异。Python 选择“读时向上查、写时就地写”，在共享与独立之间画了一条干净的分界线。

·实际问题：这个“空类当 struct 用”的技巧正是 27.13 节数据记录的基础；而“赋值不继承”这条规则则解释了所有“以为改了实例属性、结果所有实例都变了”的经典 bug——其实根源往往是写到了类属性（在类体顶层赋值）而不是实例属性。

·初学者误区：① 以为 pass 可有可无——类体必须至少一条语句，空类体语法不合法；② 以为“类必须有实例才有用”——类本身就是对象，可独立存取属性；③ 把“给类挂属性”和“给实例挂属性”搞混——rec.name = ... 是类属性，所有实例共享；④ 以为 x.name = 'Sue' 会“顺带”改掉 y——不会，只有 x 受影响。

·生活例子：类像公司的公开邮箱（rec.name），每个员工（实例）都默认看这封公告；某员工在公告下贴了张自己的便签（x.name = 'Sue'），从此他只看自己的便签——

—但公告还是那份公告，其他员工看到的还是它。

代码分析

```
>>> class rec: pass                                #
>>> rec.name = 'Pat'                                #
>>> rec.age = 40
>>> print(rec.name)                                # C struct
Pat
```

·解释器如何处理：class rec: pass 创建类对象（字典近乎为空）。rec.name = 'Pat' 与 rec.age = 40 是两条普通赋值语句，把键值写入类对象的 dict。print(rec.name) 则是“读类对象的属性”——先查 rec 自己的字典，命中即返回。整个过程没有创建任何实例，类对象独立完成存取——证明“类即对象”。

```
>>> x = rec()                                       #
>>> y = rec()
>>> x.name, y.name                                # name
('Pat', 'Pat')
```

·解释器如何处理：两个实例的字典都是空的，但 class 都指向 rec。读 x.name：x 空 → 沿链接查 rec → 命中 'Pat'。y 同理。所以两个实例“共享”同一个值——严格说不是共享，而是各自沿链接取到同一份。

```
>>> x.name = 'Sue'                                  # x
>>> rec.name, x.name, y.name
('Pat', 'Sue', 'Pat')
```

·解释器如何处理：x.name = 'Sue' 只往 x 的字典里写键 name。之后读取：rec.name 直接命中类字典 ('Pat')；x.name 命中 x 字典 ('Sue'，无需向上)；y.name 沿链接命中类字典 ('Pat')。同一个名字、三条不同的查找路

径、三个不同的结果——这就是"命名空间树 + 逐层查找"模型最直观的演示。

27.12 类：揭开引擎盖 (Classes: Under the Hood)

英文原文

In fact, as we'll explore in more detail in Chapter 29, the attributes of a namespace object are usually implemented as dictionaries, and class inheritance trees are, generally speaking, just dictionaries with links to other dictionaries. If you know where to look, you can see this explicitly.

For example, the `dict` attribute is the namespace dictionary for most classbased objects, much as in modules. Some classes may also—or instead—define attributes in slots, an advanced and seldom-used feature called slots with impacts that we'll note along the way, but whose full coverage we'll largely postpone until Chapter 32. Normally, though, `dict` literally is the attribute namespace of an instance or class.

To illustrate, the following inspects the namespaces of objects in the prior section's REPL session, filtering out built-ins in the class with a generator expression as we've done before leaving just the names we've assigned:

```
python >> list(key for key in rec.dict if not key.starts
with('')) ['name','age'] >> list(x.dict) ['name'] >> list(y
.dict) []
```

Here, the class's namespace dictionary shows the name and age attributes we assigned to it, x has its own name, and y is still empty. Because of this model, an attribute can often be fetched by either dictionary indexing or attribute notation, but only if it's present on the object in question—attribute notation kicks off inheritance search, but indexing looks in the single object only. As you'll see later, both have valid roles, and the `dir` built-in collects inherited names:

```
python >> x.name, x.dict['name'] # Attributes present here are dict keys ('Sue','Sue') >> x.age # But attribute fetch checks classes too 40 >> x.dict['age'] # Indexing dict does not do inheritance KeyError:'age' >> x.dict # Object namespace versus inheritance {'name':'Sue'} >> [attr for attr in dir(x) if attr[:2]!=''] ['age','name']
```

In addition, to facilitate inheritance search on attribute fetches, each instance has a link to its class that Python creates for us—it's called `class`, if you want to inspect it:

```
python >> x.class # Instance-to-class link
```

`<class'main.rec'>` Classes also have a `bases` attribute, which is a tuple of references to a class's superclass objects—in this example just the implicit and automatic

c object root class we'll explore later:

```
python >> rec.bases # Class-to-superclasses link (<class 'object'>)
```

These two attributes are how class trees are literally represented in memory by Python. Internal details like these are not required knowledge—class trees are implied by the code you run, and their search is normally automatic—but they can often help demystify the model.

The main point here is that Python's class model is extremely dynamic. Classes and instances are just linked namespace objects, with attributes created on the fly by assignment. Those assignments usually happen within the class statements you code, but they can occur anywhere you have a reference to one of the objects in the tree.

In fact, even methods, normally created by a `def` nested in a class, can be created completely independently of any class object. The following, for example, defines a simple function outside of any class that takes one argument:

```
python >> def uppername(obj):  
    return obj.name.upper() # Still needs a self argument  
(obj) There is nothing about a class here yet—it's a simple function, and it can be called as such at this point, provided we pass in an object obj with a name attribute.
```


tribute, whose value in turn has an upper method—our class and instances happen to fit the expected interface and kick off string uppercase conversion:

```
python >> upername(rec), upername(x), upername(y) ('PAT','SUE','PAT')
```

If we assign this simple function to an attribute of our class, though, it becomes a method, callable through any instance, as well as through the class name itself as long as we pass in an instance manually—a technique we'll leverage further in the next chapter:²

```
python >> rec.method = upername # Now it's a class's method! >> x.method() # Run method to process x'SUE'>> y.method() # Same, but pass y to self'PAT'>> rec.method(x) # Can call through instance or class'SUE'
```

Normally, classes are filled out by class statements, and instance attributes are created by assignments to self attributes in method functions. The point again, though, is that they don't have to be; OOP in Python really is mostly about looking up attributes in linked namespace objects.

中文翻译

事实上，正如我们将在第 29 章更详细地探索的那样，命名空间对象的属性通常实现为字典（dictionary），而类的继承树，一般说来，只是“带有指向其他字典的链接的字典”。只要你找对地方看，就能亲眼看到这一点。例如，dict 属性

就是大多数基于类的对象的命名空间字典，和模块很相似。有些类也可能（或者改为）在 slots 中定义属性——这是一个高级而很少使用的特性，称为槽（slots），它的影响我们沿途会提到，但完整覆盖要推迟到第 32 章。不过通常来说，dict 字面上就是实例或类的属性命名空间。为了说明，下面检查上一节 REPL 会话中各对象的命名空间，用我们之前用过的生成器表达式过滤掉类中的内建名字，只留下我们赋的值：python >>> list(key for key in rec.dict if not key.startswith('_')) ['name','age'] >>> list(x.dict) ['name'] >>> list(y.dict) [] 这里，类的命名空间字典显示了挂给它的 name 和 age 属性；x 有自己的 name；y 仍然是空的。正因为这个模型，一个属性通常既可以用字典索引获取，也可以用属性记法获取——但前提是它存在于被查的那个对象上：属性记法会触发继承搜索，而字典索引只查单个对象。你以后会看到，两者各有用途，而 dir 内建函数会收集继承来的名字：python >>> x.name, x.dict['name'] # 这里的属性就是字典键('Sue','Sue') >>> x.age # 但属性获取还会检查类40 >>> x.dict['age'] # 索引字典不做继承 KeyError:'age'>>> x.dict # 对象命名空间对比继承{'name':'Sue'} >>> [attr for attr in dir(x) if attr[:2] != '_'] ['age','name'] 此外，为了方便属性获取时的继承搜索，每个实例都有一个指向其类的链接，这个链接由 Python 替我们创建——它叫 class，如果你想检查它的话：python >>> x.class # 实例到类的链接<class'main.rec'> 类还有一个 bases 属性，它是一个元组，包含对类超类对象的引用——在这个例子中，只有隐式且自动存在的 object 根类，我们稍后会探索：python >>> rec.bases # 类到超类的链接(<class'object'>,) 这两个属性就是 Python 在内存中实际表示

类树的方式。这类内部细节并非必需知识——类树由你运行的代码隐含决定，搜索通常也是自动进行的——但它们常常有助于揭开模型的神秘面纱。这里的主要观点是：Python 的类模型极其动态。类和实例只是相互链接的命名空间对象，属性通过赋值随时创建。这些赋值通常发生在你编写的 class 语句中，但只要持有树中某个对象的引用，在任何地方都可以进行。事实上，甚至方法——通常由嵌套在类中的 def 创建——也可以完全独立于任何类对象来创建。例如，下面在任何类之外定义了一个只带一个参数的简单函数：

```
python >>> def uppername(obj): return obj.name.upper() # 仍然需要一个 self 参数 (obj) 这里还没有任何东西——它是一个简单函数，此刻就可以作为函数调用，只要传入一个带 name 属性的对象 obj，且其值又有 upper 方法——我们的类和实例恰好符合这个预期的接口，触发了字符串的大写转换：python >>> uppername(rec), uppername(x), uppername(y) ('PAT','SUE','PAT')
```

不过，如果把这个简单函数赋给类的一个属性，它就变成了方法，可以通过任何实例调用，也可以通过类名本身调用（只要手动传入一个实例）——这是下一章会进一步利用的技巧：²

```
python >>> rec.method = uppername # 现在它是类的方法了！>>> x.method() # 运行方法来处理 x'SUE'>>> y.method() # 一样，但把 y 传给 self'PAT'>>> rec.method(x) # 可以通过实例或类调用'SUE'通常，类由 class 语句填满，实例属性由方法函数中对 self 属性的赋值创建。但再次强调：它们并非必须如此；Python 中的 OOP 实际上主要就是在链接的命名空间对象中查找属性。
```

深度理解

·核心概念：本章的"引擎盖时刻"——类的所有机制可以浓缩成三样东西：`dict`（每个对象的属性字典）、`class`（实例→类的链接）、`bases`（类→超类的链接元组）。继承树 = 字典 + 链接，再无其他。

·底层实现：`x.name` 的解释器流程：查 `x.dict` → 未命中则沿 `x.class` 找到类 → 查类字典 → 未命中再沿类 `bases` 逐个查超类字典 → 全未命中抛 `AttributeError`。而 `x.dict['name']` 是纯字典索引，不做任何向上查找——所以 `x.dict['age']` 会 `KeyError`，尽管 `x.age` 能拿到 40。`dir(x)` 则是"类字典 + 所有基类字典的键"的并集，所以能看到继承来的 `age`。

·为什么这样设计：把命名空间做成字典，让"属性"与"字典键"统一——属性系统零专用机制，赋值/读取就是字典读写。这种"数据结构的复用"让整个对象模型可以用一句话向任何人讲清楚，也让内省（`introspection`）变得免费：`vars(x)`、`x.dict` 直接可看。

·实际问题：`uppername` 变方法的例子揭示了一个深刻的真相："方法"这个身份不是函数自带的，而是"函数被挂在类上"这个位置赋予的。绑定机制（方法对象自动把实例作为第一实参）由描述符协议触发，与函数是否"生来就是方法"无关。这为"把任意函数变成方法"（`monkey-patching`、装饰器包装）铺平了道路。也解释了为什么 `self` 必须显式——Python 无法在函数定义时知道它将来会不会变成方法，干脆统一显式。

·初学者误区：① 以为 dict 是给高级用户用的内部细节——它恰恰是理解本模型的钥匙；② 以为"属性记法"和"字典索引"完全等价——只有对象自己有的键才等价，继承来的不行；③ 以为方法有特殊存储——方法只是类字典里的普通函数，绑定发生在调用瞬间；④ 以为 dir 只列自己定义的属性——它列的是整条继承链的并集。

·生活例子：dict 是个人档案袋，class 是"入职公司名"，bases 是"公司归属的总部"。查 x.age 像人事流程：先翻本人档案（没有）→ 问公司制度（有，40 岁退休年龄）→ 查到即答；而 x.dict['age'] 是只翻本人档案，翻不到就说"档案里没有"。

代码分析

```
>>> list(key for key in rec.__dict__ if not key.startswith('__')...
['name', 'age']
```

·解释器如何处理：生成器表达式遍历 rec.dict（类对象的字典），过滤掉以开头的内建键，剩下的就是我们手工赋的 name 和 age。这证明"类属性"物理上就是类字典里的键值对。

```
>>> x.name, x.__dict__['name'] #
('Sue', 'Sue')
>>> x.age #
40
>>> x.__dict__['age'] #
KeyError: 'age'
>>> [attr for attr in dir(x) if attr[:2] != '__']
['age', 'name']
```

·解释器如何处理：x.dict['name'] 与 x.name 结果相同，

因为 name 就在 x 自己的字典里。x.age 能取到 40，是因为属性记法沿 class 链接找到了类字典里的 age；而 x.dict['age'] 是纯索引，x 字典里没有 age，直接 KeyError——同样的属性，两种取法，结果一成一败。dir(x) 走的是类型层级，把类里的 age 和实例里的 name 合并返回。

```
>>> def uppername(obj):
    return obj.name.upper() # self obj
>>> rec.method = uppername #
>>> x.method() # x
'SUE'
>>> rec.method(x) #
'SUE'
```

·解释器如何处理：uppername 本是模块里的普通函数。rec.method = uppername 把它存入类字典——从这一刻起，通过实例访问 x.method 时，描述符协议看到"类字典里存的是函数"，自动把它包装成绑定方法（记住 x 作为第一实参），所以 x.method() 等价于 uppername(x)。而 rec.method(x) 走的是"类本身取属性"的路径（函数没有被绑定），需要手动传实例。同一个函数对象，两种调用姿势——绑定与否，只看访问它时经过了谁。

27.13 回顾记录：类对比字典 (Records Revisited: Classes Versus Dictionaries)

英文原文

Although the simple classes of the prior section are meant to illustrate class model basics, the techniques they employ can also be used for real work. For example, Chapters 8 and 9 showed how to use dictionaries, tuples, and lists to record properties of entities in our programs, generically called records.

It turns out that classes can often serve better in this role—they package information like dictionaries, but can also bundle processing logic in the form of methods. For reference, here is an example for tuple- and dictionary-based records we used earlier in the book (using one of many dictionary coding techniques):

```
python >> rec = ('Pat', 40.5, ['dev','mgr']) # Tuple-based record
>> print(rec[0]) Pat
>> rec = {}
>> rec['name'] = 'Pat' # Dictionary-based record
>> rec['age'] = 40.5 # Or {...}, dict(n=v), etc.
>> rec['jobs'] = ['dev','mgr']
>> print(rec['name']) Pat
```

As we just saw, though, there are also multiple ways to do the same with classes. Perhaps the simplest is this—trading keys for attributes:

```
python >> class rec: pass
>> rec.name = 'Pat' # Class-based record
>> rec.age = 40.5
>> rec.jobs = ['dev','mgr']
>> print(rec.name) Pat
```

This code has substantially less syntax than the dictionary equivalent. It uses an empty class statement to generate an empty namespace object, which we then fill out by assigning class attributes over time, as bef

ore. This works, but a new

class statement will be required for each distinct record we will need. Perhaps more typically, we can instead generate instances of an empty class to represent each distinct entity:

```
python >> class rec: pass >> pers1 = rec() # Instance
-based records >> pers1.name = 'Bob' >> pers1.jobs
= ['dev','mgr'] >> pers1.age = 40.5 >> pers2 = rec()
>> pers2.name = 'Sue' >> pers2.jobs = ['dev','cto'] >>
pers1.name, pers2.name ('Bob','Sue')
```

Here, we make two records from the same class. Instances start out life empty, just like classes. We then fill in the records by assigning to attributes. This time, though, there are two separate objects, and hence two separate name attributes. In fact, instances of the same class don't even have to have the same set of attribute names; in this example, one has a unique age name.

Instances really are distinct namespaces, so each has a distinct attribute dictionary. Although they are normally filled out consistently by a class's methods, they are more flexible than you might expect.

Finally, we might instead code a more full-blown class to implement the record and its processing—something that data-oriented dictionaries do not directly support:


```
python >> class Person:
```

```
def init(self, name, jobs, age=None): # class = data +  
    logic self.name = name self.jobs = jobs self.age = ag  
e def info(self): return (self.name, self.jobs) >> rec1 =  
Person('Bob', ['dev','mgr'], 40.5) # Construction calls  
>> rec2 = Person('Sue', ['dev','cto']) >> rec1.jobs, rec  
2.info() # Attributes + methods (['dev','mgr'], ('Sue', ['  
dev','cto']))
```

This scheme also makes multiple instances, but the class is not empty this time: we've added logic (methods) to initialize instances at construction time and collect attributes into a tuple on request. The constructor imposes some consistency on instances here by always setting the name, job, and age attributes, even though the latter can be omitted when an object is made.

Together, the class's methods and instance attributes create a package, which combines both data and logic.

We could further extend this code by adding logic to compute salaries, parse names, and so on. Ultimately, we might link the class into a larger hierarchy to inherit and customize an existing set of methods via the automatic attribute search of classes, or perhaps even store instances of the class in a file with Python object pickling to make them persistent.

In fact, we will—in the next chapter, we'll expand on this analogy between classes and records with a more realistic running example that demonstrates class basics in action.

To be fair to other tools, in this form, the two preceding class construction calls more closely resemble dictionaries made all at once, but still seem less cluttered and provide extra processing methods. In fact, the class's construction calls more closely resemble Chapter 9's named tuples—which makes sense, given that named tuples really are classes with extra logic to map attributes to tuple offsets:

```
python >> rec = dict(name='Pat', jobs=['dev','mgr'],  
age=40.5) # Dictionaries >> rec = {'name':'Pat','jobs':  
['dev','mgr'],'age': 40.5} # Ditto >>... setup code... >>  
rec = Rec('Pat', ['dev','mgr'], 40.5) # Named tuples
```

In the end, although types like dictionaries and tuples are flexible, classes allow us to add behavior to objects in ways that built-in types and simple functions do not directly support. Although we can store functions in dictionaries too (e.g., under key `info` to mimic the class's method), using them to process instances is nowhere near as natural and structured as it is in classes, and key lookup has no notion of inheritance to enable customization.

But to vet the purported benefits of classes firsthand, you'll have to move ahead to the next chapter.

中文翻译

虽然上一节的简单类是用来演示类模型基础的，但它们采用的技术也可以用于真实工作。例如，第 8 章和第 9 章展示了如何使用字典、元组和列表来记录程序中实体的属性，统称为记录 (record)。事实证明，类在这个角色上往往更胜一筹——它们像字典一样打包信息，还能以方法的形式捆绑处理逻辑。作为参考，这里给出我们早先在书中用过的元组版和字典版记录示例（使用众多字典编码技巧之一）：

```
python >>> rec = ('Pat', 40.5, ['dev','mgr']) # 基于元组的记录
>>> print(rec[0]) Pat
>>> rec = {}
>>> rec['name'] = 'Pat' # 基于字典的记录
>>> rec['age'] = 40.5 # 或者 {...}、dict(n=v) 等
>>> rec['jobs'] = ['dev','mgr']
>>> print(rec['name']) Pat
```

不过，正如我们刚刚看到的，用类做同样的事情也有多种方式。最简单的也许是这样——把键换成属性：

```
python >>> class rec: pass
>>> rec.name = 'Pat' # 基于类的记录
>>> rec.age = 40.5
>>> rec.jobs = ['dev','mgr']
>>> print(rec.name) Pat
```

这段代码比字典版少了很多语法。它用一个空的 class 语句生成一个空的命名空间对象，然后像之前一样，通过随时间给类属性赋值来填充它。这能工作，但每需要一个不同的记录，就得写一条新的 class 语句。更典型的做法或许是：生成一个空类的实例来表示每个不同的实体：

```
python >>> class rec: pass
>>> pers1 = rec() # 基于实例的记录
>>> pers1.name = 'Bob'
>>> pers1.jobs = ['dev','mgr']
>>> pers1.age = 40.5
>>> pers2 = rec()
>>> pers2.name = 'Sue'
>>> pers2.jobs = ['dev','cto']
>>> pers1.name, pers2.name ('Bob','Sue')
```

这里，我们用同一个类制作了两条

记录。实例和类一样，以空的身份开始生命。然后我们通过给属性赋值来填充记录。但这次是两个独立的对象，因而有两条独立的 name 属性。事实上，同一个类的实例甚至不必拥有相同的属性名集合；在这个例子中，一条记录有独一无二的 age 名字。实例确实是各自独立的命名空间，所以每个都有独立的属性字典。虽然它们通常由类的方法一致地填充，但它们比你想象的更灵活。最后，我们也可以编写一个更完整的类，来实现记录及其处理——这是面向数据的字典无法直接支持的东西：

```
python >>> class Person:
def init(self, name, jobs, age=None): # 类 = 数据 + 逻辑
self.name = name
self.jobs = jobs
self.age = age
def info(self):
return (self.name, self.jobs)
>>> rec1 = Person('Bob', ['dev','mgr'], 40.5) # 构造调用
>>> rec2 = Person('Sue', ['dev','cto'])
>>> rec1.jobs, rec2.info() # 属性 + 方法(['dev','mgr'], ('Sue', ['dev','cto']))
```

这个方案同样制作多个实例，但这次类不是空的：我们添加了逻辑（方法），在构造时初始化实例，并可按需把属性收集进元组。构造函数通过总是设置 name、job 和 age 属性，为实例施加了一定的一致性——尽管最后一个参数在创建对象时可以省略。类的方法和实例属性合在一起，构成了一个包（package），把数据与逻辑结合起来。我们还可以进一步扩展这段代码，加入计算薪水、解析名字等逻辑。最终，我们可以把类链接到更大的层次结构中，借助类的自动属性搜索来继承和定制一组现有方法；甚至可以用 Python 的对象 pickle（腌制/序列化）把类的实例存进文件，让它们持久化。事实上，我们确实会这么做——在下一章，我们会用一个更真实、运行中的示例，展开“类与记录”的类比，演示类基础的实际应用。公平地说，在这种形式下，前面两个类的

构造调用更像是"一次性做成的字典", 但它们看起来仍然不那么杂乱, 而且提供了额外的处理方法。事实上, 类的构造调用更接近第 9 章的命名元组 (named tuple) ——这很合理, 因为命名元组其实就是带有"把属性映射到元组偏移量"的额外逻辑的类: `python >>> rec = dict(name='Pat', jobs=['dev','mgr'], age=40.5) # 字典`
`>>> rec = {'name':'Pat','jobs': ['dev','mgr'],'age': 40.5} # 同上`
`>>> ...`
设置代码 ... `>>> rec = Rec('Pat', ['dev','mgr'], 40.5) #`
命名元组归根结底, 虽然字典和元组这类类型很灵活, 但类允许我们以内建类型和简单函数无法直接支持的方式为对象添加行为。虽然我们也可以把函数存进字典 (比如放在 `info` 键下模仿类的方法), 但用它们去处理隐含的实例, 远不如在类中来得自然和结构化; 而且字典键查找没有继承的概念, 无法实现定制。但要亲手验证类的所谓优点, 你还得继续往前走, 到下一章去。

深度理解

·核心概念: 本节把前几节的技术"实战化": 类可以当记录 (record) 用, 且有一个字典永远给不了的东西——行为。
·三种方案递进: 类属性当字段 (共享版) → 实例属性当字段 (每实体独立) → 完整类 (字段 + 方法, 构造时统一初始化)。

·底层实现: 三种方案在内存中的差异一目了然: 方案一数据全在类字典; 方案二数据分散在各自实例的 `dict` (`pers1` 有 `name/jobs/age`, `pers2` 只有 `name/jobs`——属性集合都可以不一样, 因为实例字典互不相关); 方案三实例字典在 `init` 执行后一次性被写满, 且 `info` 方法留在类

里共享。

·设计原因：为什么"空类当 struct"能行？因为 Python 的属性赋值无门槛——字典能加键，对象就能加属性。这比 C 的 struct 更自由（字段无须预声明），比 Pascal 的 record 更自由。而给记录配方法，则实现了"数据内聚行为"——调用方拿到对象就知道能对它做什么。

·实际问题：现实工程中这类模式的演进路径是：裸字典（手滑键名易错）→ 类记录（属性自动补全、可加方法）→ dataclass/namedtuple（现代 Python 的标准答案，自动生成 init/repr）。Lutz 此处埋的"记录=类"的类比，正是后续学习数据类（dataclass）的思想起点。

·初学者误区：① 以为"类的实例必须由类的 init 统一初始化"——不一定，先建空实例再逐属性赋值完全合法（动态性）；② 以为同一类的实例"结构必须相同"——上例 pers1 有 age 而 pers2 没有，照样运行；③ 以为字典能完全替代类——字典存不了行为与继承，键拼错还不报错（rec['nam'] 静默 KeyError 也罢了，赋值时 rec['nam'] = ... 直接造出新键，bug 更难找）。

·生活例子：字典版记录是一张手写信息卡（只能看字）；实例版记录是带贴纸的文件夹（每人的贴纸可以不同）；完整类记录是标准简历表 + 填表规则（init 规定必填项，info 是"一键导出摘要"的按钮）。面试官（调用方）只看到表就能知道能做什么——这就是"数据+逻辑打包"的价值。

代码分析

```
>>> class rec: pass
>>> pers1 = rec()                                #
>>> pers1.name = 'Bob'
>>> pers1.jobs = ['dev', 'mgr']
>>> pers1.age = 40.5
>>> pers2 = rec()
>>> pers2.name = 'Sue'
>>> pers2.jobs = ['dev', 'cto']
>>> pers1.name, pers2.name
('Bob', 'Sue')
```

解释器如何处理：pers1 = rec() 与 pers2 = rec() 创建两个互不相关的实例（各自 dict 为空，class 都指向 rec）。随后的属性赋值分别写入各自的字典。注意 pers1 多了一个 age 键而 pers2 没有——解释器对此毫无意见，实例字典本来就可以形态各异。最后读取 pers1.name、pers2.name 各自命中自己的字典，返回 ('Bob','Sue')。

```
>>> class Person:
    def __init__(self, name, jobs, age=None):    # = +
        self.name = name
        self.jobs = jobs
        self.age = age
    def info(self):
        return (self.name, self.jobs)
>>> rec1 = Person('Bob', ['dev', 'mgr'], 40.5)    #
>>> rec2 = Person('Sue', ['dev', 'cto'])
>>> rec1.jobs, rec2.info()                        # +
(['dev', 'mgr'], ('Sue', ['dev', 'cto']))
```

解释器如何处理：Person('Bob', ['dev','mgr'], 40.5) 触发构造流程：分配空实例 → 自动调用 init(实例,'Bob', [...], 40.5) → 三个 self.x = ... 把实例字典写满 → 返回。rec

2 只传了两个位置参数，age 用默认值 None——默认参数机制照常生效。rec1.jobs 是实例字典直接命中；rec2.info() 则沿 class 找到类里的 info 函数并绑定 rec2，函数内部用 self.name、self.jobs 组装元组返回。数据（属性）+ 逻辑（方法）在此合体为“包”。

27.14 章节总结 (Chapter Summary)

英文原文

This chapter introduced the basics of coding classes in Python. We studied the syntax of the class statement and learned how to use it to build up a class inheritance tree. We also studied how Python automatically fills in the first argument in method functions, how attributes are attached to objects in a class tree by simple assignment, and how specially named operator-overloading methods intercept and implement built-in operations for our instances (e.g., expressions and printing). Now that we've explored the mechanics of classes in Python, the next chapter turns to a larger and more realistic example that ties together much of what we've learned about OOP so far and introduces some new topics. After that, we'll continue our look at class coding, taking a second pass over the model to fill in some of the details that were omitted here to keep things simple. First, though, let's work through a quiz to review the basics we've covered so far.

中文翻译

本章介绍了用 Python 编写类的基础知识。我们学习了 `class` 语句的语法，以及如何使用它搭建类的继承树。我们还学习了 Python 如何自动填充方法函数中的第一个参数、如何通过简单赋值把属性挂接到类树中的对象上，以及特殊命名的运算符重载方法如何拦截并为我们的实例实现内建操作（如表达式和打印）。既然我们已经探索了 Python 中类的机制，下一章将转向一个更大、更真实的示例，把目前学到的 OOP 知识串联起来，并引入一些新主题。之后，我们会继续学习类编码，对模型进行第二轮扫视，补上那些为了保持简单而省略的细节。不过首先，让我们做一个小测验，复习一下目前学过的要点。

27.15 自测：Quiz（随章测验）

测验题目（中英对照）

1. How are classes related to modules? 类与模块有什么关系？
2. How are instances and classes created? 实例和类是如何创建的？
3. Where and how are class attributes created? 类属性在哪里、如何创建？
4. Where and how are instance attributes created? 实例属性在哪里、如何创建？
5. What does `self` mean in a Python class? 在 Python 类中，`self` 是什么意思？
6. How is operator overloading coded in a Python class? Python 类中如何编写运算符重载？
7. When might you want to support operator overloading in your classes? 什么时候你可能想让

自己的类支持运算符重载？ 8. Which operator-overloading method is most commonly used? 哪个运算符重载方法最常用？ 9. What are the three key concepts required to understand Python OOP code? 理解 Python OOP 代码所需的三个关键概念是什么？

参考答案

1. Classes are always nested inside a module; they are attributes of a module object. Classes and modules are both namespaces, but classes correspond to statements (not entire files) and support the OOP notions of multiple instances, inheritance, and operator overloading (modules mostly do not). In a sense, a module is like a single-instance class, without inheritance, which corresponds to an entire file of code. 类总是嵌套在模块内部；它们是模块对象的属性。类和模块都是命名空间，但类对应的是语句（而不是整个文件），并支持 OOP 的多个实例、继承和运算符重载等概念（模块大多不支持）。在某种意义上，模块就像一个没有继承的“单实例类”，对应着整个代码文件。 2. Classes are made by running class statements; instances are created by calling a class as though it were a function. 类通过运行 class 语句创建；实例通过像调用函数一样调用类来创建。 3. Class attributes are created by assigning attributes to a class object. They are normally generated by top-level assignments nested in a class statement—each name assigned in the class statement block becomes an attribute of the class object (technically, the class statement'

s local scope morphs into the class object's attribute namespace, much like a module). Class attributes can also be created, though, by assigning attributes to the class anywhere a reference to the class object exists—even outside the class statement.类属性通过对类对象赋值属性来创建。它们通常由嵌套在 class 语句中的顶层赋值生成——class 语句块中赋值的每个名字都会成为类对象的属性（严格说，class 语句的局部作用域会蜕变为类对象的属性命名空间，很像模块）。不过，类属性也可以在任何持有类对象引用的地方通过赋值创建——甚至可以在 class 语句之外。

4. Instance attributes are created by assigning attributes to an instance object. They are normally created within a class's method functions coded inside the class statement, by assigning attributes to the self argument (which is always the implied instance). Again, though, they may be created by assignment anywhere a reference to the instance appears, even outside the class statement. Usually, all instance attributes are initialized in the init constructor method; that way, later method calls can assume the attributes already exist.实例属性通过对实例对象赋值属性来创建。它们通常在 class 语句内部编写的方法函数中，通过给 self 参数（它始终是隐含的实例）赋值来创建。但同样，它们也可以在出现实例引用的任何地方通过赋值创建，甚至在 class 语句之外。通常，所有实例属性都在 init 构造函数方法中初始化；这样，后续的方法调用就可以假定这些属性已经存在。

5. self is the name commonly given to the first (leftmost) argument in a class's method function; Python

n automatically fills it in with the instance object that is the implied subject of the method call. This argument need not be called self (though this is a very strong convention); its position is what is significant. (Ex-C++ or Java programmers might prefer to call it this because in those languages that name reflects the same idea; in Python, though, this argument must always be explicit in method headers and code.) self 是类的方法函数中第一个（最左边）参数常用的名字；Python 会自动用作为方法调用隐含主语的那个实例对象来填充它。这个参数不一定非得叫 self（虽然这是非常强的约定）；重要的是它的位置。（前 C++ 或 Java 程序员可能更喜欢叫它 this，因为在那些语言里这个名字反映同样的概念；但在 Python 中，这个参数在方法头和方法代码里必须始终显式写出。）

6. Operator overloading is coded in a Python class with specially named methods; they all begin and end with double underscores to make them unique. These are not built-in or reserved names; Python just runs them automatically when an instance appears in the corresponding operation. Python itself defines the mappings from operations to special method names. 运算符重载在 Python 类中通过特殊命名的方法编写；这些方法都以双下划线开头和结尾，以保证唯一。它们不是内建或保留名字；当实例出现在对应操作中时，Python 只是自动运行它们。从操作到特殊方法名的映射由 Python 自己定义。

7. Operator overloading is useful to implement objects that resemble built-in types (e.g., sequences or numeric objects such as matrixes), and to mi

mic the built-in type interface expected by a piece of code. Mimicking built-in type interfaces enables you to pass in class instances that also have state information (i.e., attributes that remember data between operation calls). You generally shouldn't use operator overloading when a simple named method will suffice, though.运算符重载对实现"类似内建类型"的对象（如序列或矩阵之类的数值对象）很有用，也可以用来模仿某段代码所期望的内建类型接口。模仿内建类型接口，使你可以传入还带有状态信息（即在操作调用之间记住数据的属性）的类实例。不过，当简单的命名方法足够用时，一般不应该使用运算符重载。

8. The `__init__` constructor method is the most commonly used; almost every class uses this method to set initial values for instance attributes and perform other startup tasks. The `__str__` method (and its repr sibling) is not uncommon, but not as much of a fixture as `__init__`. `__init__` 构造函数方法最常用；几乎每个类都用它来为实例属性设置初始值并执行其他启动任务。`__str__` 方法（及其孪生 `__repr__`）并不罕见，但不像 `__init__` 那样是标配。

9. The special `self` argument in method functions, the inheritance search for attributes, and the `__init__` constructor method are the cornerstones of OOP code in Python. If you get these, you should be able to read the text of most OOP Python code—apart from these, it's largely just packages of functions. In classes, `self` represents the automatic object argument, and `__init__` is commonly used to initialize instances.方法函数中特殊的 `self` 参数、属性的继承搜索、以及 `__init__` 构造函数方法，是 Python

OOP 代码的三大基石。掌握了它们，你就能读懂大多数 Python OOP 代码的正文——除此之外，它们大体上只是函数的包（packages of functions）。在类中，self 代表自动传入的对象参数，init 常用于初始化实例。

原书脚注

1. Not to be confused with the init.py files in module packages! The method here is a class constructor function used to initialize the newly created instance, not a namespace for a module-package folder. See Chapter 24 for more details. 不要与模块包中的 init.py 文件混淆！这里的方法是用作类构造函数、初始化新建实例的函数，而不是模块包文件夹的命名空间。更多细节见第 24 章。

2. In fact, this is one of the reasons the self argument must always be explicit in Python methods—because methods can be created as simple functions independent of a class, they need to include the implied instance argument explicitly. They can be called as either functions or methods, and Python can neither guess nor assume that a simple function might eventually become a class's method. The main reason for the explicit self argument, though, is to make the meanings of names more apparent: names not referenced through self are simple variables mapped to scopes, while names referenced through self with attribute notation are obviously instance attributes. 事实上，这正是 self 参数在 Python 方法中必须始终显式的原因之一——因为方法可以独立于类、作为简单函数创建，它们需要显式地

包含这个隐含的实例参数。它们既可以作为函数调用，也可以作为方法调用，而 Python 既无法猜测、也无法假设一个简单函数最终可能成为类的方法。不过，显式 `self` 参数的主要原因是让名字的含义更清楚：不通过 `self` 引用的名字是映射到作用域的普通变量，而通过 `self` 用属性记法引用的名字显然就是实例属性。

本章总结

技术拓展 (Technical Expansion)

1. 实际项目中的应用场景

·领域模型 (Domain Model)：本章的"记录"类比直接通向现代工程实践——员工、订单、商品等实体用类建模（或更现代的 `dataclass`），属性存状态，方法存行为。所有 Web 框架 (Django 的 `Model`、SQLAlchemy 的 `Declarative Base`) 都是"类=数据库表"模式的体现。

·方法绑定与回调：理解"方法=挂到类上的函数 + 绑定机制"，你才能看懂装饰器 (`decorator`)、`functools.partial`、以及 `pytest fixtures`、Flask 路由等框架魔法——它们大量利用"函数被包装后改变调用签名"的技术。

·运算符重载的真实舞台：应用代码中你很少自己写 `add`，但你天天消费别人写的——NumPy 数组的 `a + b`、pandas 的 `Series`、`pathlib` 的 `Path / "sub"`。学会本章的分派机制，读框架源码不再神秘。

· init 作为"属性清单": 作者说"看 init 就知道实例有什么属性"——这是真实工程中的实用技巧: 读陌生类时先看 init, 几秒钟就能建立对象的字段画像。

2. 与其他语言的区别 (Java/C++)

维度	Python	Java	C++
类的创建	class 是运行时可执行语句	编译期声明 + 字节码类文件	编译期声明
类的本质	带 dict/class/bases 的运行时	元数据对象 (java.lang.Class)	无运行时实体 (除 RTTI 表)
实例属性声明	无需声明, 运行时赋值即创建	必须预声明字段	必须预声明成员
self/this	显式 self, 位置即语义	隐式 this 指针	隐式 this
方法定义	类体内的普通 def (本质是函数赋值)	专用方法语法 + 可见性修饰符	专用成员函数语法
继承搜索	运行期逐层字典查找	编译期解析 + 虚表 (vtable)	编译期解析 + 虚表分派
运算符重载	通过特殊命名方法 (add 等), 可选	不支持	通过 operator + 成员/友元函数
对象唯一性控制	无 (多继承默认允许, 菱形用 MRO 解)	单继承 (类) + 多接口	多继承 (菱形需虚继承)

核心洞察: Python 把"类"彻底降格为普通数据对象 (字典+链接), 因此动态性 (运行时加属性、函数变方法、运行时改类) 是免费的; Java/C++ 为了编译期类型安全, 付出了类系统僵硬、运行期不可改的代价。这也是为什么 Python 框架能玩出"动态注册方法"等魔术而 Java 需要依赖反射框架。

3. Python 发展历史背景

- Python 1.0 (1994) : Guido 借鉴 Smalltalk 与 C++ 设计出 Python 的类模型, 但坚持两条自己的原则: 一切属性可动态添加 (Smalltalk 的做法), self 显式 (独此一家, 理由是"名字意义更明显、函数可独立于类存在")。
- classic classes → new-style classes (2000-2008) : Python 2.2 引入"新式类" (继承 object), 统一了类型与类的模型; Python 3 中只有新式类——本章 `rec.bases` 显示 `<class'object'>` 正是这个历史演进的遗迹。
- 为什么 init 不叫 constructor: Python 的设计者刻意区分"构造" (new, 分配) 与"初始化" (init, 填充状态) ——这是为了支持不可变对象等高级用法, 也解释了为什么语言文档从不说 init 分配内存。
- 现代演进: dataclasses (3.7)、typing 的 NamedTuple 等工具把本章"手工填属性"的模式自动化, 但底层机制一字未变——理解本章, 你就理解了现代 Python 框架的基石。

4. 高级开发者需要掌握的相关知识

- 描述符协议 (descriptor protocol) : 方法绑定只是描述符 get 的一个应用; property、classmethod、static method、slots 全是描述符。第 38 章详细展开, 这是从"会用"到"懂 Python 对象模型"的必经之路。
- 元类 (metaclass) : type 是类的类——"class 语句执

行时发生了什么"的最终答案。第 40 章才讲，但知道"类对象由 type 创建"已足以解释本章所有现象。

- MRO 与 C3 线性化：多继承时 bases 不再只有一个元素，查找顺序由 `type(obj).mro()` 决定。第 31 章内容，遇到"菱形继承"前先了解概念即可。

- 特殊方法查找的"类型层"特性：`add`、`str` 等特殊方法是通过类型查找的（不走实例字典）——这意味着 `x.add = ...` 这样给实例挂特殊方法无效。这是高级调试中常踩的暗坑。

- slots：用固定属性名换取内存与访问速度，但会改变对象的动态性。第 32 章覆盖。

- 序列化与持久化：`pickle` 能存类实例，靠的正是"实例 = dict + class"这个结构——本章 27.12 节的引擎盖视角是理解它的直接前提。

学习建议 (Learning Advice)

- 重要程度：★★★★★ (5/5) 。本章是整个 OOP 部分的语法基石——`self`、属性、继承、`init` 四大概念在此全部登场，后续第 28-32 章全部建立在本章机制之上。这是全书最重要的章节之一。

- 应该掌握到什么程度：

- 必会（烂熟于心）：能默写"实例属性 = 对 `self` 赋值，类属性 = 类体顶层赋值"；能解释 `x.setdata(v)` 为什么等价于 `FirstClass.setdata(x, v)`；能画出 27.1 节那种"实例→

类→超类"的查找链路；会用 `init` 初始化实例；能说清"读属性做继承搜索、写属性只落当前对象"。

·熟练：写一个带 `str/add` 的小类；能用 `dict/class/bases/dir` 检查任何对象。

·了解：运算符重载的哲学（可选、模仿内建语义、主要用于工具类）；`repr` 与 `str` 的区分（下章细讲）。

·学习方法（关键）：本章 27.5、27.7、27.10 三个示例请亲手在 REPL 里逐行敲一遍，并主动做破坏性实验：在调用 `setdata` 前调用 `display`（看报错）、给 `y` 也赋个 `name`（看是否影响 `x`）、把 `rec.method` 赋给实例试试（验证特殊方法不走实例字典的对比）。亲手撞过错误，机制才真正内化。

·后续学习内容：

·立即：第 28 章"A More Realistic Example"——把本章机制用于完整程序，并引入 `repr`、运算符重载的更多实战

。

·近程：第 29 章（命名空间/作用域细节）、第 30 章（运算符重载全览、`imul` 等）、第 31 章（多继承与 MRO）

。

·进阶（学完本书后）：`dataclasses` 文档、`functools.partial` 源码、`inspect` 模块的 `signature/ismethod`——用"引擎盖"视角审查日常框架代码。

·一句话总结：Python 的 OOP 就三句话——类是可执行语句造出的命名空间对象，实例是带着类链接的空命名空间，方法就是挂在类上、调用时自动把实例塞进第一个参

数的函数。其余一切，都是这三句话的推论。

本章完。下一章：A More Realistic Example（一个更真实的示例）